TEST_Deloitte

September 26, 2022

0.1 Question 1

A water supply company asks Deloitte for an analysis that allows senior management to understand the internal and external factors that affect water demand in their catchment area, as well as to make a forecast of such demand for the following two years. Therefore, your Manager asks you to develop a descriptive and predictive analysis of water demand by region based on historical data and external variables.

In the Excel find attached you will find a data set that contains data related to the water demand/consumption (i.e. cubic meters), price, weather and client characteristics (i.e. number of households and companies). With this data, perform the tasks that your Manager requested and explain the methodology chosen and results briefly. In addition, we encourage you to make any comments related to the data set provided (e.g. limitations, missing variables, etc.)

[1]:
```
import numpy as np
import pandas as pd
import seaborn as sns

import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)
```

[2]:
```
# Needs openpyxl lib
data_path = 'DTT_Data_Scientist_Test.xlsx'
database = pd.read_excel(data_path)
database.head()
```

[2]:
```
  Zona    Fecha         MC  Clientes  Precio_Medio  Precipitacion  \
0    A  2019 1Q  531930.18  16650.12      1.701437       1.755423
1    A  2019 2Q  557170.62  16658.13      1.689908       0.790385
2    A  2019 3Q  569989.20  16735.56      1.645398       0.537554
3    A  2019 4Q  556886.13  16743.57      1.700518       0.862167
4    A  2020 1Q  513545.34  16761.37      1.740904       2.760333

   Temperatura  Poblacion  Desempleo       Renta  …  Empresas_Construccion  \
0     9.952688    41281.8    2863.35  6158.25000  …                    339
1    20.306114    41281.8    2641.80  6158.25000  …                    339
2    25.323083    41281.8    2592.45  6158.25000  …                    339
3    11.566897    41281.8    2636.55  6158.25000  …                    339
4     8.468011    41517.0    2638.65  6322.05945  …                    344
```

```
   Empresas_CTH  Empresas_Servicios  Hogar_Total  Hogar_1pax  Hogar_2pax  \
0          1110                1691        16041        4381        4634
1          1110                1691        16051        4384        4637
2          1110                1691        16143        4409        4663
3          1110                1691        16151        4411        4666
4          1071                1750        16133        4406        4660

   Hogar_3pax  Hogar_4pax  Hogar_5pax  Hogar_6pax
0        3200        3012         597         217
1        3202        3014         597         217
2        3221        3031         601         218
3        3222        3033         601         218
4        3219        3029         601         218

[5 rows x 24 columns]
```

[3]:
```
database.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 72 entries, 0 to 71
Data columns (total 24 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   Zona                   72 non-null     object
 1   Fecha                  72 non-null     object
 2   MC                     72 non-null     float64
 3   Clientes               72 non-null     float64
 4   Precio_Medio           72 non-null     float64
 5   Precipitacion          72 non-null     float64
 6   Temperatura            72 non-null     float64
 7   Poblacion              72 non-null     float64
 8   Desempleo              72 non-null     float64
 9   Renta                  72 non-null     float64
 10  Densidad               72 non-null     float64
 11  Edad_Media             72 non-null     float64
 12  Empresas_Total         72 non-null     int64
 13  Empresas_Industria     72 non-null     int64
 14  Empresas_Construccion  72 non-null     int64
 15  Empresas_CTH           72 non-null     int64
 16  Empresas_Servicios     72 non-null     int64
 17  Hogar_Total            72 non-null     int64
 18  Hogar_1pax             72 non-null     int64
 19  Hogar_2pax             72 non-null     int64
 20  Hogar_3pax             72 non-null     int64
 21  Hogar_4pax             72 non-null     int64
 22  Hogar_5pax             72 non-null     int64
```

```
 23  Hogar_6pax             72 non-null     int64
dtypes: float64(10), int64(12), object(2)
memory usage: 13.6+ KB
```

Semantic Interpretation The following description also shows interesting questions about possible mistakes that must be treated.

1. Zona: geographic region.
2. Fecha: year and trimester (1Q: Jan,Feb,Mar; 2Q:Apr,May,Jun, etc.)
3. MC: amount of demand (cubic meters)
4. Clientes: number of clients. (**Why is not an Integer?**)
5. Precio_Medio: mean price for each MC.
6. Precipitacion: meteorology measurement related to rain, humidity, etc. (**No information about the metric unit, or if it is an average for the period or just an accumulative quantity**)
7. Temperature: average wether
8. Poblacion: Population (**why is not an integer**)
9. Desempleo: Unemployment (**why is not an integer**).
10. Renta: average housing price.
11. Densidad: population density
12. Edad_Media: average age of the population
13. Empresas_Total: total number of companies in the zone
14. Empresas_x: total number of specific type of company ($\Sigma_x Empresas_x =^? Empresas_{Total}$)
15.
16.
17.
18. Hogar_Total: total number of homes.
19. Hogar_x: total number of home for a given type (1, 2,...,6 habitants)($\Sigma_x Hogar_x =^? Hogar_{Total}$)
20.
21.
22.
23.
24.

[4]:
```
# Row sample
database.iloc[0]
```

[4]:
```
Zona                              A
Fecha                       2019 1Q
MC                        531930.18
Clientes                   16650.12
Precio_Medio               1.701437
Precipitacion              1.755423
Temperatura                9.952688
Poblacion                   41281.8
Desempleo                   2863.35
Renta                       6158.25
Densidad                   2.206639
Edad_Media                43.965158
Empresas_Total                 3603
Empresas_Industria              463
Empresas_Construccion           339
Empresas_CTH                   1110
Empresas_Servicios             1691
Hogar_Total                   16041
Hogar_1pax                     4381
Hogar_2pax                     4634
Hogar_3pax                     3200
Hogar_4pax                     3012
Hogar_5pax                      597
Hogar_6pax                      217
Name: 0, dtype: object
```

Checking and Cleaning

[5]:
```
features_zero = []
for c in database.columns:
    if not database[database[c]== 0].all().all():
        features_zero.append(c)
features_zero
```

[5]:
```
['Empresas_Total',
 'Empresas_Industria',
 'Empresas_Construccion',
 'Empresas_CTH',
 'Empresas_Servicios',
 'Hogar_6pax']
```

Are these zero-values a real quantity? or Are they missed values?

Checking the original table, we can find a `-` character. I think it refers a 'no information'.

Also these missings are correlated with the geographic zone (`Zona`):

[6]:
```
for c in features_zero:
    print(set(database[database[c]== 0]['Zona']))
```

```
{'E'}
{'E'}
{'E'}
{'E'}
{'E'}
{'C', 'F'}
```

- the zone `E` has not information about Companies (`Empresas_*`) and
- The zones `C`, `F` have no information about Homes 6pax Homes (`Hogar_6pax`).

One possible interpretation is as follow: * It is not very common to find a zone without any company (industrial, services, comercial, etc) * It could be possible to find zones with no 6pax homes (for example, neighborhoods with small apartments) This interpretation suggest the `0-values` as a missing for the feature `Empresas_*` but 0 for `Hogar_6pax`.

However, we will need to analyze it better. For example, comparing the energy demand of the zone.

[7]:
```
if database.isnull().any().sum() == 0:
    print('Not null values.')
else:
    print('Missed values found.')
```

```
Not null values.
```

Lets check the sumatory of the different features respect to its sub-type.

[8]:
```
sum_ = database[['Empresas_Industria', 'Empresas_Construccion', 'Empresas_CTH',␣
↪'Empresas_Servicios']].sum(axis=1)
(database['Empresas_Total'] == sum_).all()
```

[8]: True

[9]:
```
sum_ = database[['Hogar_1pax', 'Hogar_2pax', 'Hogar_3pax', 'Hogar_4pax',␣
↪'Hogar_5pax', 'Hogar_6pax']].sum(axis=1)
(database['Hogar_Total'] == sum_).all()
```

[9]: True

Rounding and Casting continuous to discrete

[10]:
```
#database['Poblacion'] = database['Poblacion'].astype(int)  # Truncate.
database['Poblacion'] = database['Poblacion'].round(0).astype(int)  #Rounding
database['Desempleo'] = database['Desempleo'].round(0).astype(int)
database['Clientes'] = database['Clientes'].round(0).astype(int)
```

[11]:
```
# Seasonal dates (yyyy{1,2,3,4}Q)
def fiscal_format(s):
    y,q= s.split()
    return ''.join([y, q[1], q[0]])
```

```
database['Fecha']= database['Fecha'].apply(lambda r: fiscal_format(r)) \
                                     .apply(lambda r: pd.Period(r, freq='Q'))
```

[12]:
```
database['Fecha']
```

[12]:
```
0     2019Q1
1     2019Q2
2     2019Q3
3     2019Q4
4     2020Q1
       …
67    2020Q4
68    2021Q1
69    2021Q2
70    2021Q3
71    2021Q4
Name: Fecha, Length: 72, dtype: period[Q-DEC]
```

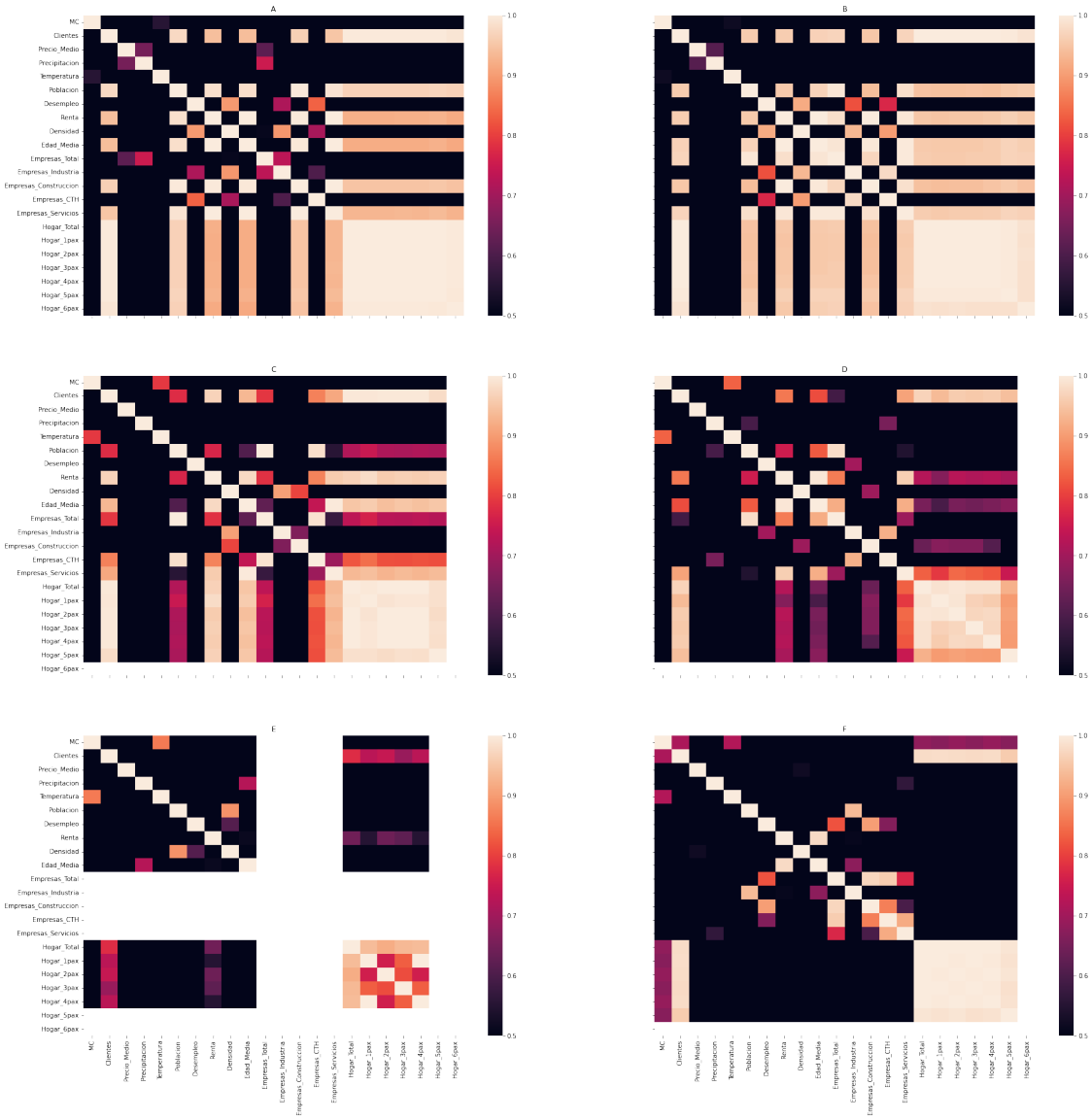
[13]:
```
database.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 72 entries, 0 to 71
Data columns (total 24 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   Zona                   72 non-null     object
 1   Fecha                  72 non-null     period[Q-DEC]
 2   MC                     72 non-null     float64
 3   Clientes               72 non-null     int64
 4   Precio_Medio           72 non-null     float64
 5   Precipitacion          72 non-null     float64
 6   Temperatura            72 non-null     float64
 7   Poblacion              72 non-null     int64
 8   Desempleo              72 non-null     int64
 9   Renta                  72 non-null     float64
 10  Densidad               72 non-null     float64
 11  Edad_Media             72 non-null     float64
 12  Empresas_Total         72 non-null     int64
 13  Empresas_Industria     72 non-null     int64
 14  Empresas_Construccion  72 non-null     int64
 15  Empresas_CTH           72 non-null     int64
 16  Empresas_Servicios     72 non-null     int64
 17  Hogar_Total            72 non-null     int64
 18  Hogar_1pax             72 non-null     int64
 19  Hogar_2pax             72 non-null     int64
 20  Hogar_3pax             72 non-null     int64
 21  Hogar_4pax             72 non-null     int64
```

```
 22  Hogar_5pax              72 non-null     int64
 23  Hogar_6pax              72 non-null     int64
dtypes: float64(7), int64(15), object(1), period[Q-DEC](1)
memory usage: 13.6+ KB
```

Correlation by Zones.

[14]:
```
from matplotlib import pyplot as plt

fig,axn = plt.subplots(3, 2, sharex=True, sharey=True)

fig.set_figheight(30)
fig.set_figwidth(30)


zones = database['Zona'].unique()
for i, ax in enumerate(axn.flat):
    ax.set_title(zones[i])
    sns.heatmap(database[(database['Zona']==zones[i])].corr(),
                vmin=.5,
                xticklabels=True, yticklabels=True,
                ax=ax);
```



There are some clear correlations between population, density, houses, etc. As a first view, it seems difficult to find a quick relationship between the companies (`Empresas_*`) and the other variables.

There are some possibilities to fill these values: * Perform a supervised approach, using all the variables as a input and `Empresas_*` as a target to predict them for Zone E could be interesting. But with a higher cost than a MC prediction (?) * Fill the variables with zero, assuming is not a missing and considering they refer just a small residential zone. * The supervised approach must be simple and fast. Doing this, it is likely to obtain some similar values than the mean or median.

- Which is the general distribution of these features?
- Which is the relationship of these features with respect to population/density/MC?

Descriptive information by Zone

[15]:
```
def feature_by_zone(feature, database, remove=''):
    zones = list(database['Zona'].unique())
    if remove:
        zones.remove(remove)
    df = {}
    for z in zones:
        df[z] = database[(database['Zona']==z)].
 ↪sort_values(by='Fecha')[feature].values

    return pd.DataFrame(df)

df = feature_by_zone('Empresas_Total', database, 'E')
total_mean = df.mean()
df.describe()
```

[15]:

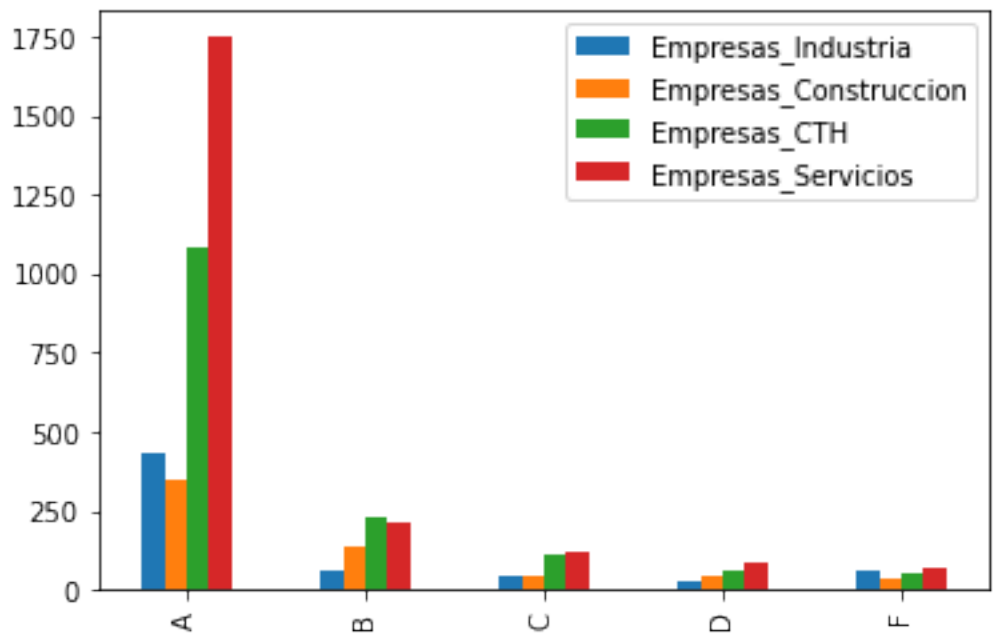
	A	B	C	D	F
count	12.000000	12.000000	12.000000	12.000000	12.000000
mean	3613.000000	628.666667	317.333333	209.333333	209.000000
std	21.759011	10.421423	2.741378	2.461830	6.149649
min	3594.000000	618.000000	315.000000	206.000000	201.000000
25%	3594.000000	618.000000	315.000000	206.000000	201.000000
50%	3603.000000	626.000000	316.000000	211.000000	211.000000
75%	3642.000000	642.000000	321.000000	211.000000	215.000000
max	3642.000000	642.000000	321.000000	211.000000	215.000000

[16]:
```
df = {}
for feat in ['Empresas_Industria', 'Empresas_Construccion', 'Empresas_CTH',␣
 ↪'Empresas_Servicios']:
    df[feat] = feature_by_zone(feat, database, 'E').mean()

pd.DataFrame(df)
```

[16]:

	Empresas_Industria	Empresas_Construccion	Empresas_CTH	Empresas_Servicios
A	436.000000	344.666667	1079.666667	1752.666667
B	58.000000	136.333333	226.000000	208.333333
C	46.666667	40.666667	114.333333	115.666667
D	22.000000	42.000000	62.333333	83.000000
F	57.000000	33.000000	52.333333	66.666667

[17]:
```
pd.DataFrame(df).plot(kind='bar');
```



[18]:
```
pd.DataFrame(df).mean()
```

[18]:
```
Empresas_Industria       123.933333
Empresas_Construccion    119.333333
Empresas_CTH             306.933333
Empresas_Servicios       445.266667
dtype: float64
```

Using the mean could bias the number of Empresas considering the zone A as an outlier. If the region E and A have similar characteristics could be a good option (we will check this).

For simplicity, we could use the mean without consideration of the outlier-Zone to see how it looks.

[19]:
```
imputation = pd.DataFrame(df).loc[['B', 'C', 'D', 'F']].mean()
imputation
```

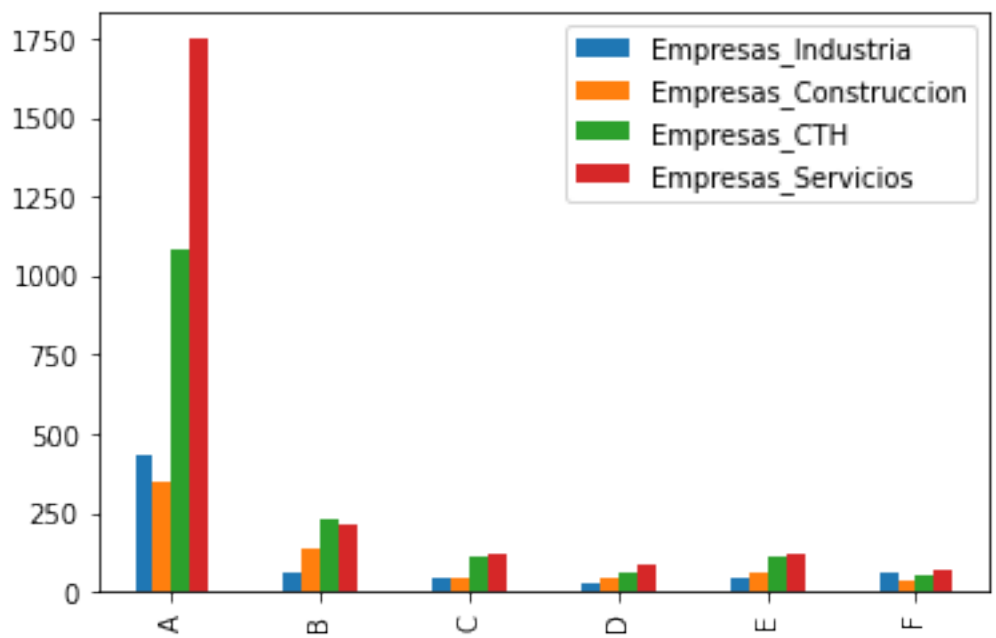
[19]:
```
Empresas_Industria        45.916667
Empresas_Construccion     63.000000
Empresas_CTH             113.750000
Empresas_Servicios       118.416667
dtype: float64
```

[20]:
```
# Copy for checking. I will chekc naother information before to decide if I use␣
↪this imputaion or not.
database_imputed = database.copy()

for feat in imputation.index:
```

```
    row_idxs = database_imputed[database['Zona'] == 'E'][feat].index
    col_idxs = database_imputed.columns.get_loc(feat)
    database_imputed.iloc[row_idxs, col_idxs] = imputation.loc[feat].item()
```

[21]:
```
row_idxs = database_imputed[database['Zona'] == 'E']['Empresas_Total'].index
col_idxs = database_imputed.columns.get_loc('Empresas_Total')
database_imputed.iloc[row_idxs, col_idxs] = imputation.sum().item()
```

[22]:
```
df = {}
for feat in ['Empresas_Industria', 'Empresas_Construccion', 'Empresas_CTH',␣
 ↪'Empresas_Servicios']:
    df[feat] = feature_by_zone(feat, database_imputed, '').mean()

pd.DataFrame(df).plot(kind='bar');
```



This plot shows how the `Empresas_*` variables looks after the mean imputation with now consideration of the Zone A.

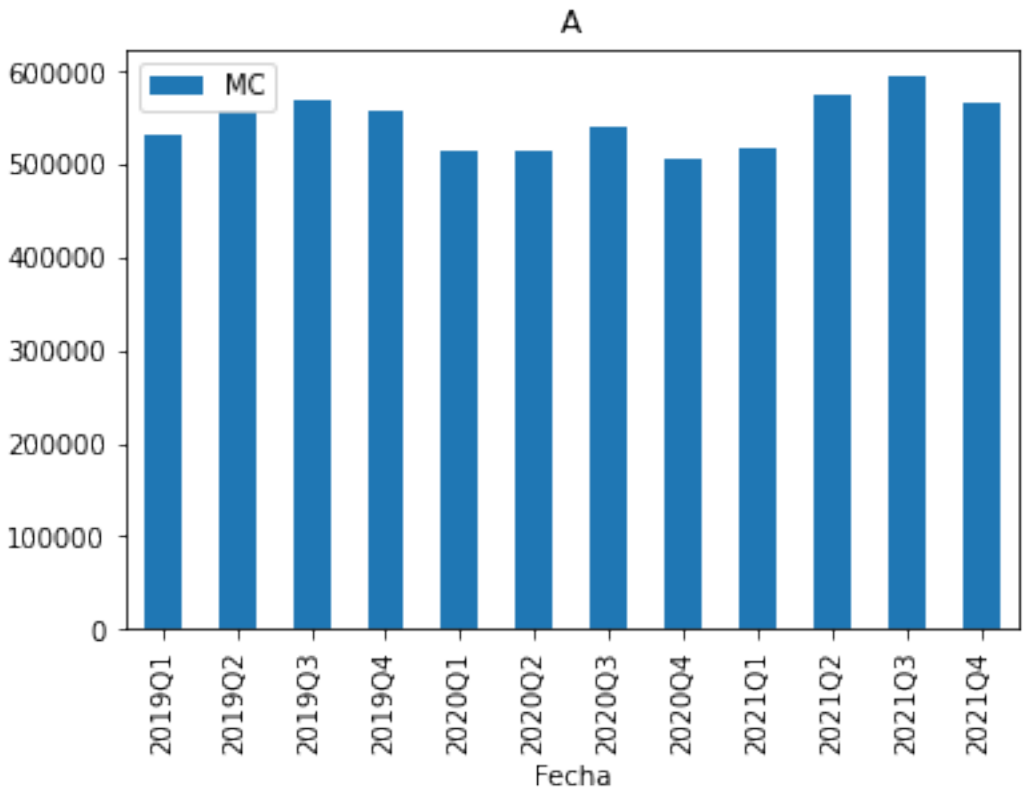
This option could be possible but the MC of the zone E is considerabily lower respect to all the others (see bellow).

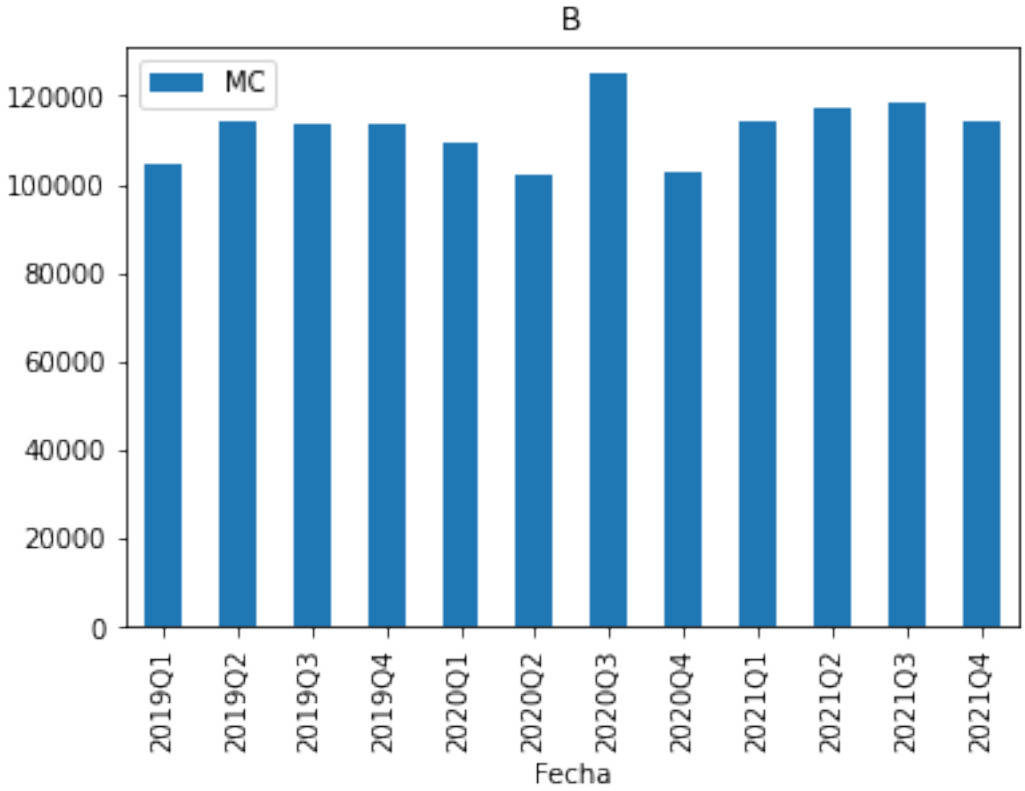
[23]:
```
(database['Clientes'] <= database['Empresas_Total'] + database['Hogar_Total']).
 ↪all()
```

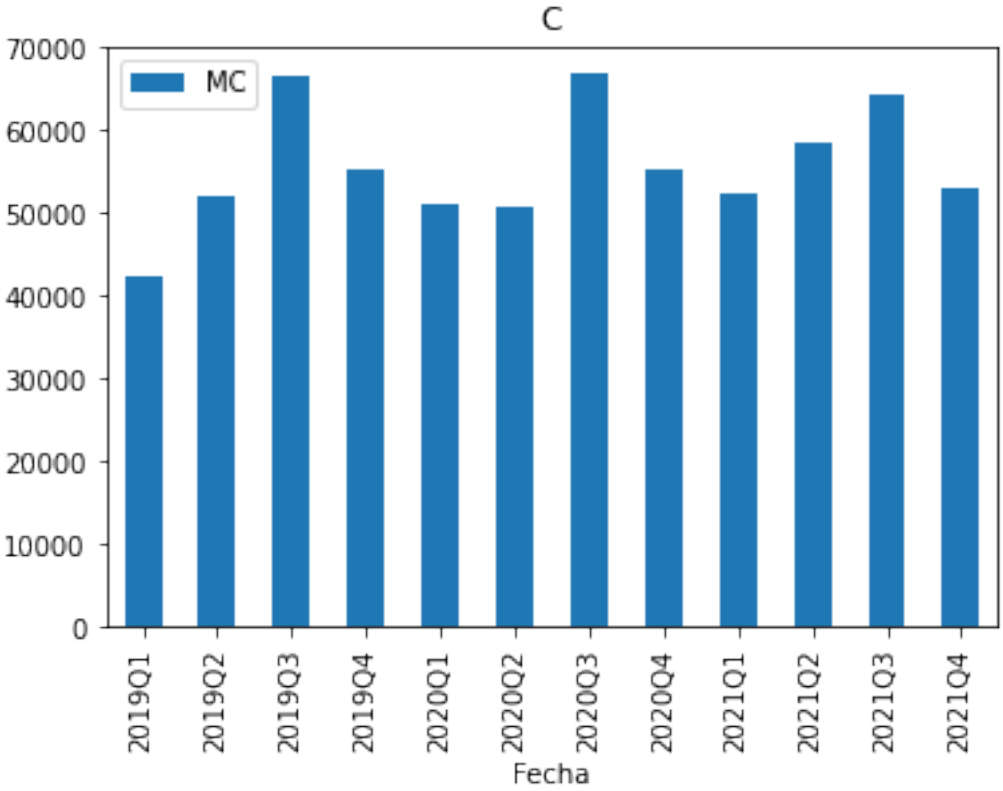
[23]: `True`

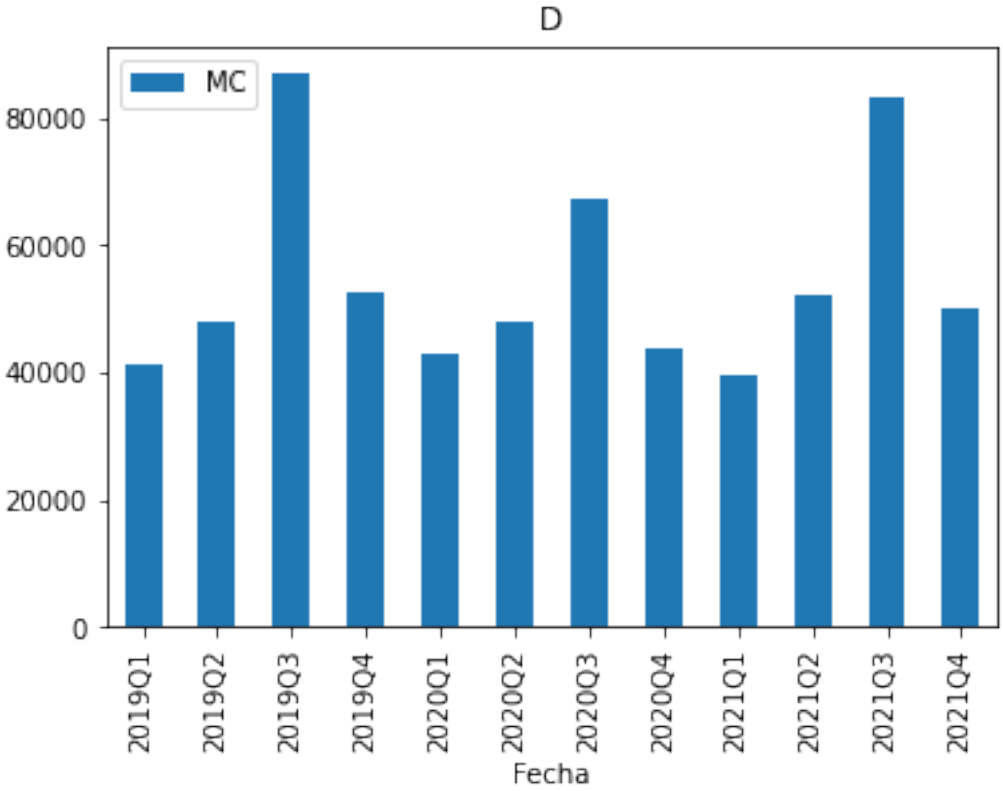
Descriptive information by Zone and Period respect to the MC

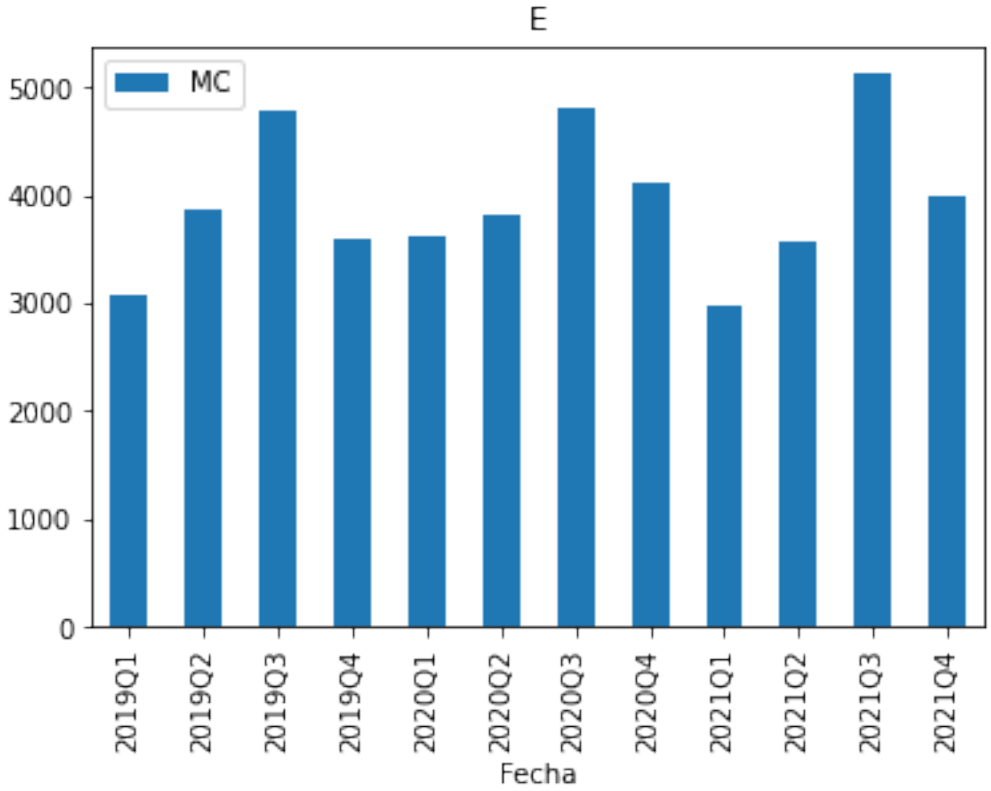
[24]:
```
for zona in database['Zona'].unique():
    database[database['Zona']==zona].sort_values(by='Fecha')[['Fecha','MC']].
↪plot(kind='bar', x='Fecha', y='MC', title=zona)
```

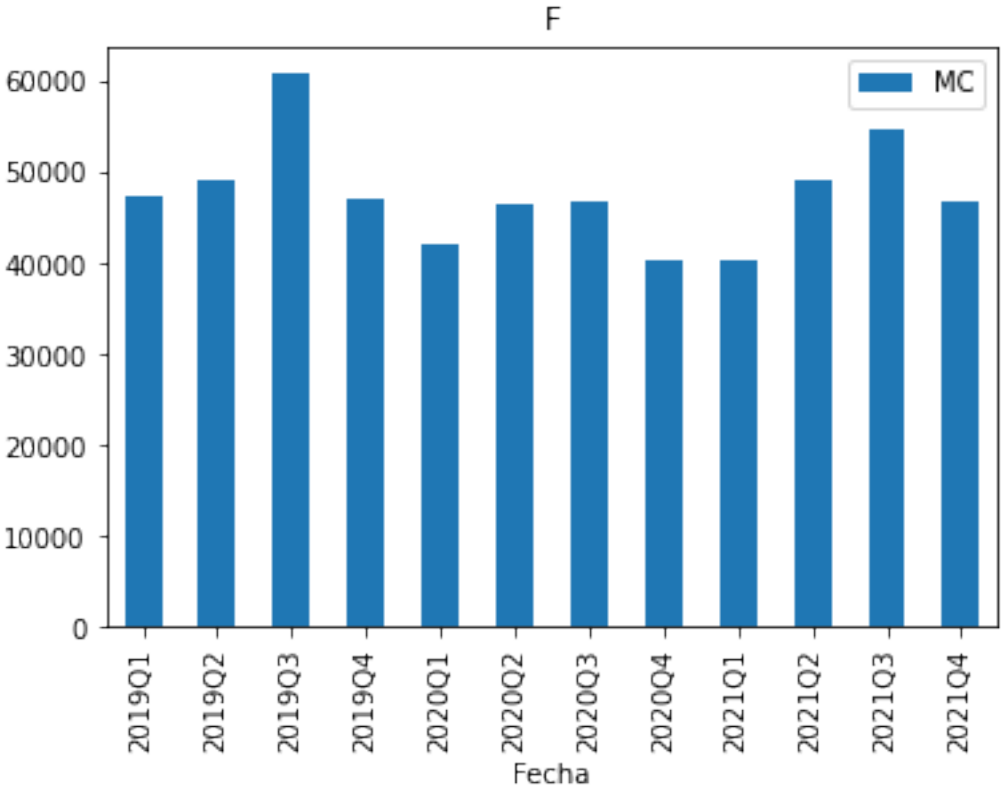



B
MC
120000
100000
80000
60000
40000
20000
0
2019Q1
2019Q2
2019Q3
2019Q4
2020Q1
2020Q2
2020Q3
2020Q4
2021Q1
2021Q2
2021Q3
2021Q4
Fecha


C
MC
70000
60000
50000
40000
30000
20000
10000
0
2019Q1
2019Q2
2019Q3
2019Q4
2020Q1
2020Q2
2020Q3
2020Q4
2021Q1
2021Q2
2021Q3
2021Q4
Fecha


D
MC
80000
60000
40000
20000
0
2019Q1
2019Q2
2019Q3
2019Q4
2020Q1
2020Q2
2020Q3
2020Q4
2021Q1
2021Q2
2021Q3
2021Q4
Fecha

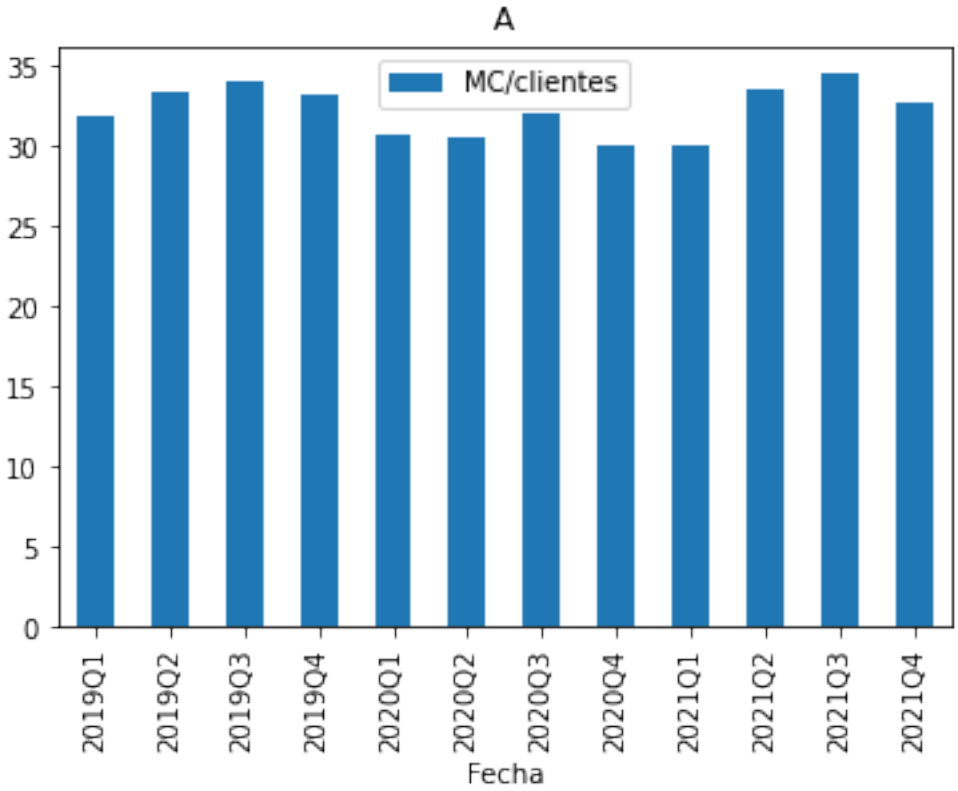

E
MC
5000
4000
3000
2000
1000
0
2019Q1
2019Q2
2019Q3
2019Q4
2020Q1
2020Q2
2020Q3
2020Q4
2021Q1
2021Q2
2021Q3
2021Q4
Fecha

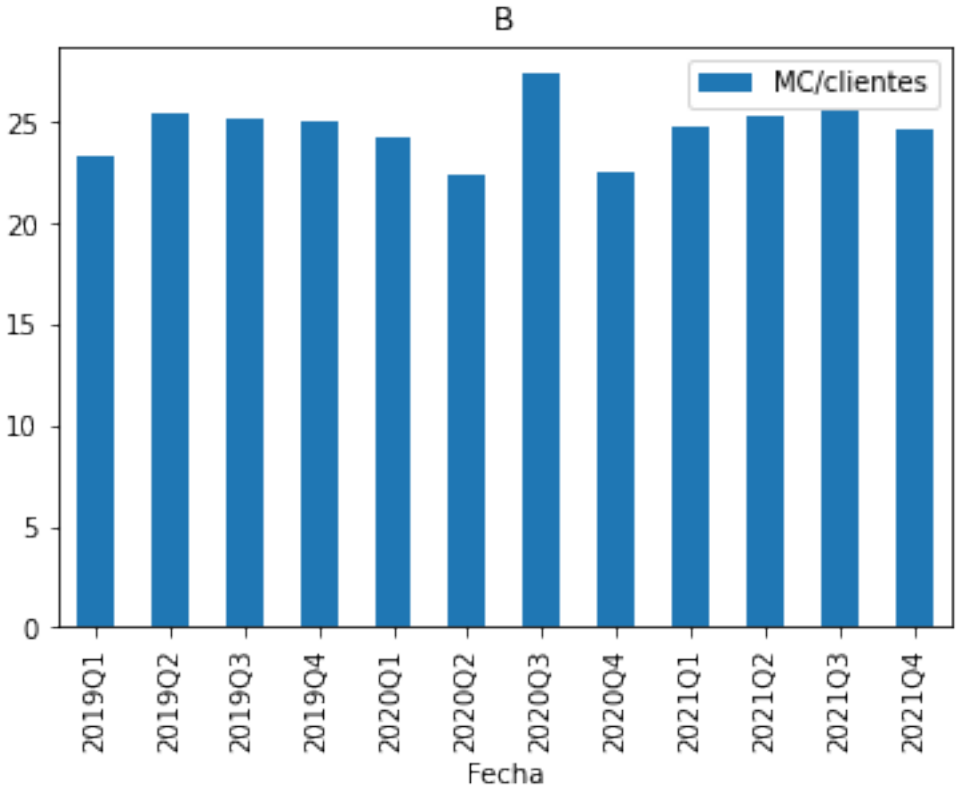


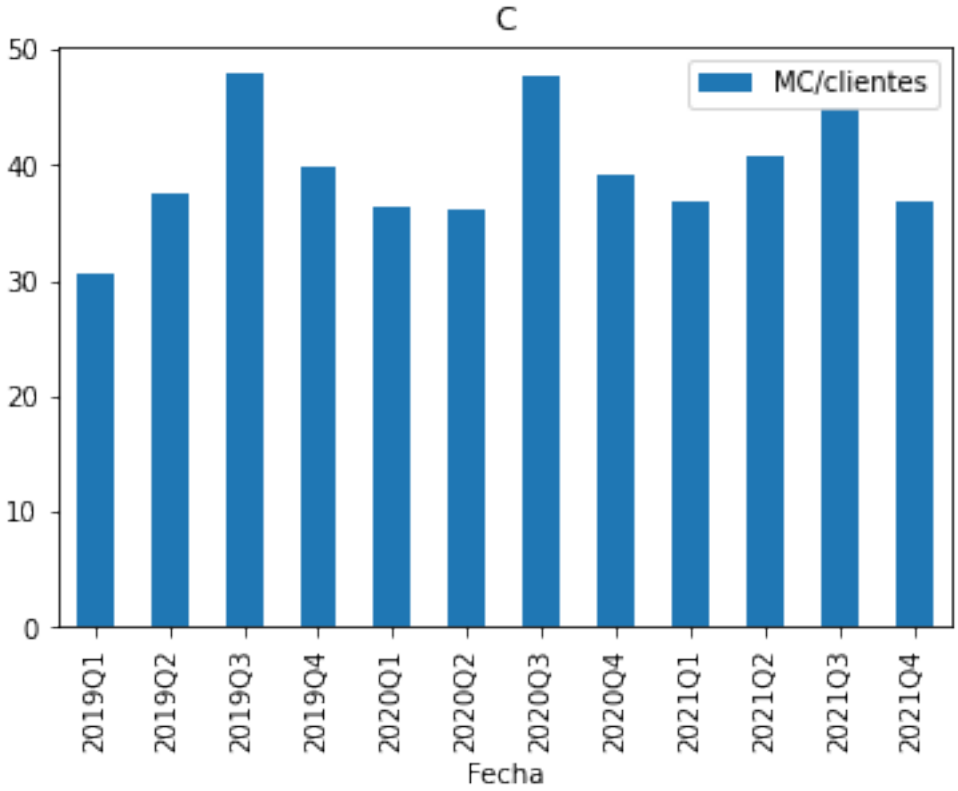
Descriptive information by Zone and respect to the demand by client (MC/Client)

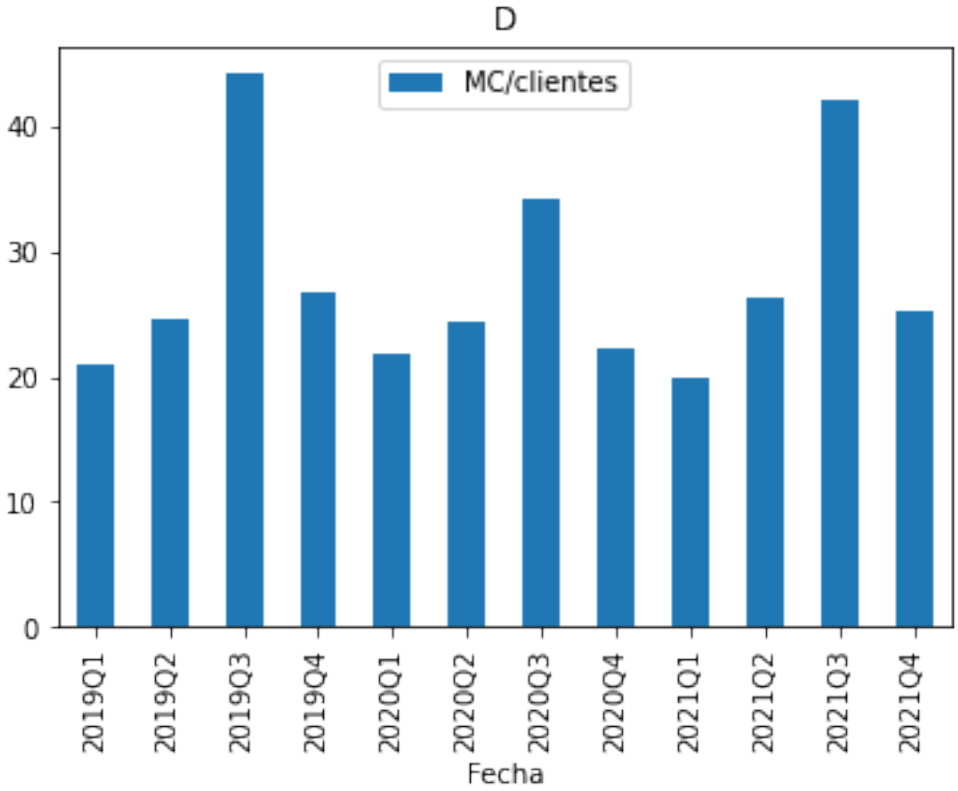
[25]:
```
mc_clientes = database[['Zona', 'Fecha', 'MC', 'Clientes']]
mc_clientes['MC/clientes'] = mc_clientes['MC'] / mc_clientes['Clientes']

for zona in mc_clientes['Zona'].unique():
    mc_clientes[mc_clientes['Zona']==zona].sort_values(by='Fecha').
 ↪plot(kind='bar', x='Fecha', y='MC/clientes', title=zona)
```

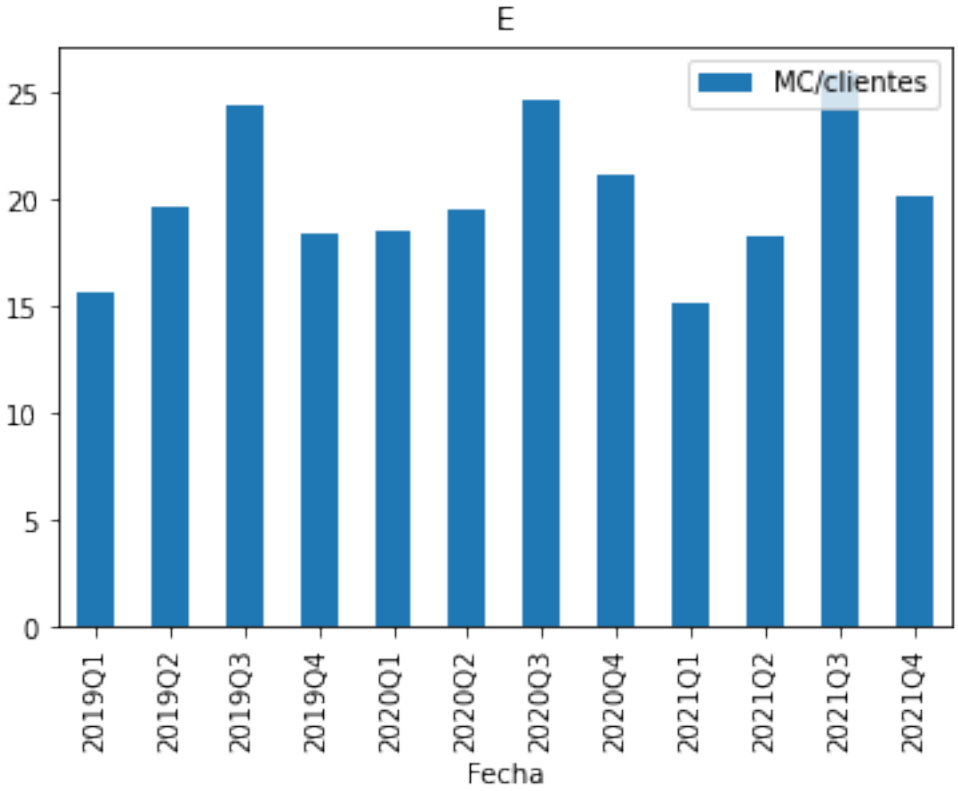
```
<ipython-input-25-7341fb4f36a2>:2: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  mc_clientes['MC/clientes'] = mc_clientes['MC'] / mc_clientes['Clientes']
```

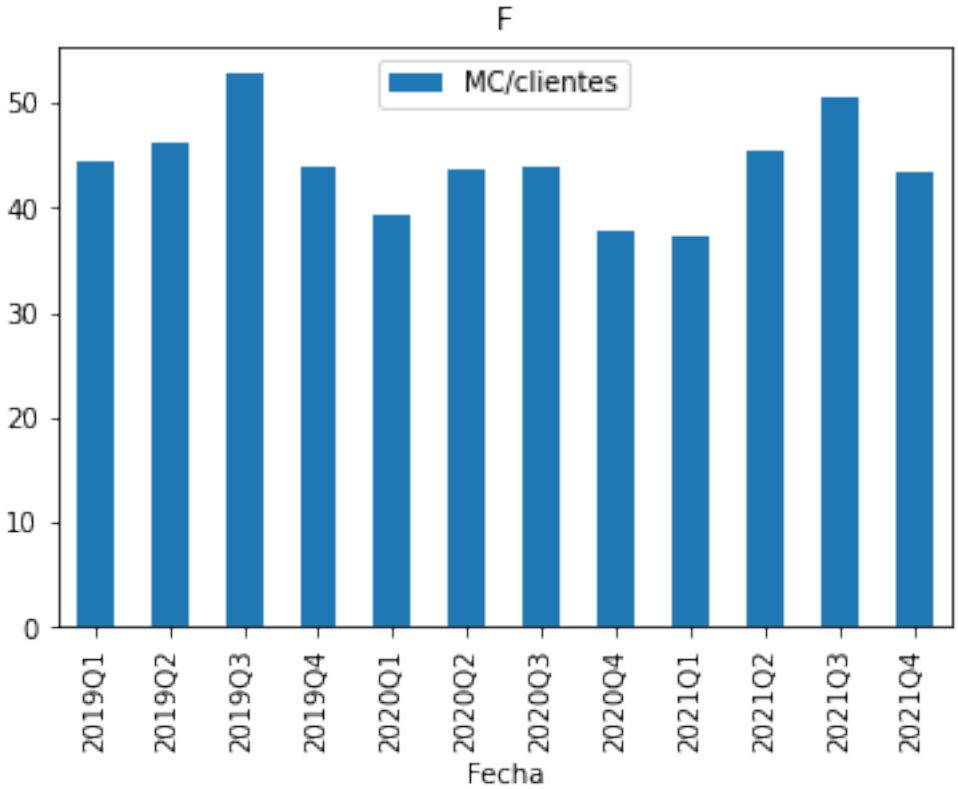

A
MC/clientes
35
30
25
20
15
10
5
0
2019Q1
2019Q2
2019Q3
2019Q4
2020Q1
2020Q2
2020Q3
2020Q4
2021Q1
2021Q2
2021Q3
2021Q4
Fecha


B
MC/clientes
25
20
15
10
5
0
2019Q1
2019Q2
2019Q3
2019Q4
2020Q1
2020Q2
2020Q3
2020Q4
2021Q1
2021Q2
2021Q3
2021Q4
Fecha


C
MC/clientes
50
40
30
20
10
0
2019Q1
2019Q2
2019Q3
2019Q4
2020Q1
2020Q2
2020Q3
2020Q4
2021Q1
2021Q2
2021Q3
2021Q4
Fecha


D
MC/clientes
40
30
20
10
0
2019Q1
2019Q2
2019Q3
2019Q4
2020Q1
2020Q2
2020Q3
2020Q4
2021Q1
2021Q2
2021Q3
2021Q4
Fecha


E
MC/clientes
25
20
15
10
5
0
2019Q1
2019Q2
2019Q3
2019Q4
2020Q1
2020Q2
2020Q3
2020Q4
2021Q1
2021Q2
2021Q3
2021Q4
Fecha



- Similar **trends** when we perform a correction with the number of clients (Ups and Downs in similar seasons).
- Then, similar peaks in similar seasons (correlation between the season and the demand(MC))
- Different variability of demand with respect to the zones: A, B shows more uniform demand than the others (more predictable?)
- Beyond the total MC demand, there are interesting views when correcting by number of clients. For example, Zone B shows less per-client demand than other zones like F. ¿Is there any particular client that biases the total demand?

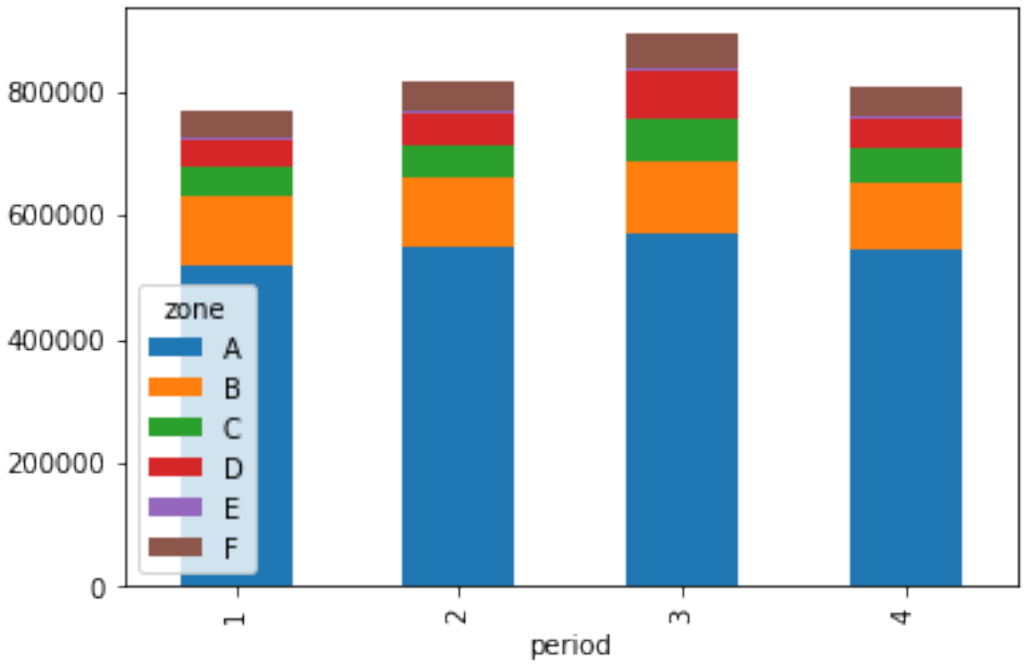
Period of demand and Zones: relative plot

[26]:
```
zones, periods, avgs = [], [], []
for zona in database['Zona'].unique():
    for q in [1,2,3,4]:
            zone_df = database[database['Zona']==zona]
            period_idx = (zone_df['Fecha'].apply(lambda r: r.quarter) == q)

            zones.append(zona)
            periods.append(q)
            avgs.append(zone_df[period_idx]['MC'].mean().item())

MC_period = pd.DataFrame({'zone': zones,
```

```
                                'period': periods,
                                'avg': avgs})
```

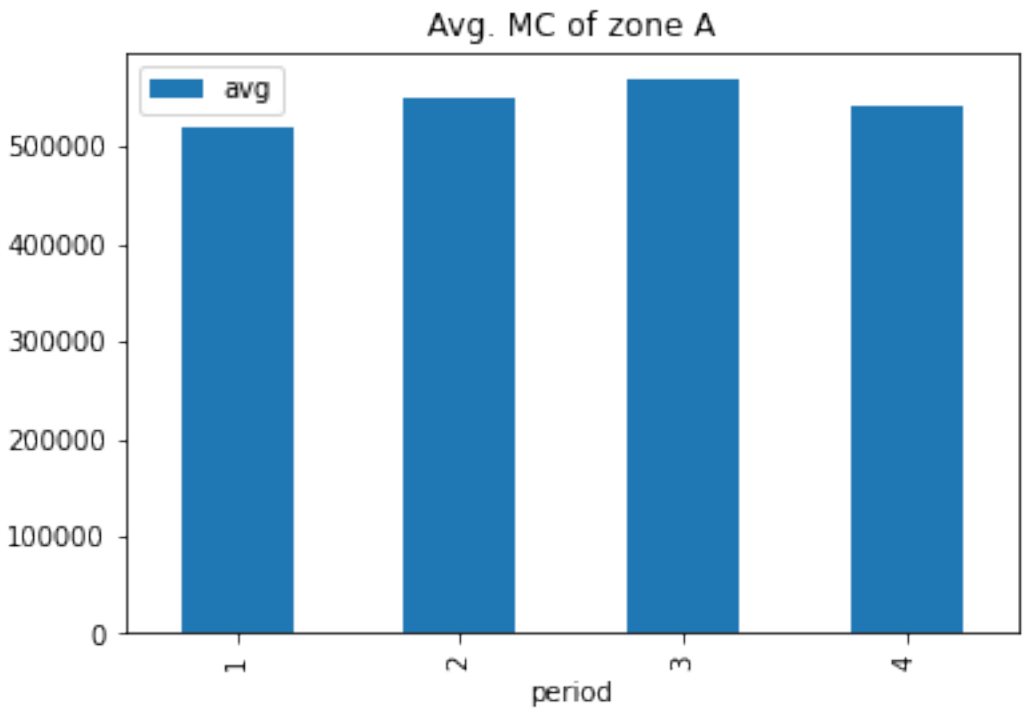
[27]:
```
MC_period.sort_values(by='avg').pivot_table(index=['period'], columns='zone',␣
↪values='avg').plot(kind='bar', stacked=True);
print('Relationship between each zone and their average demand per season.')
```

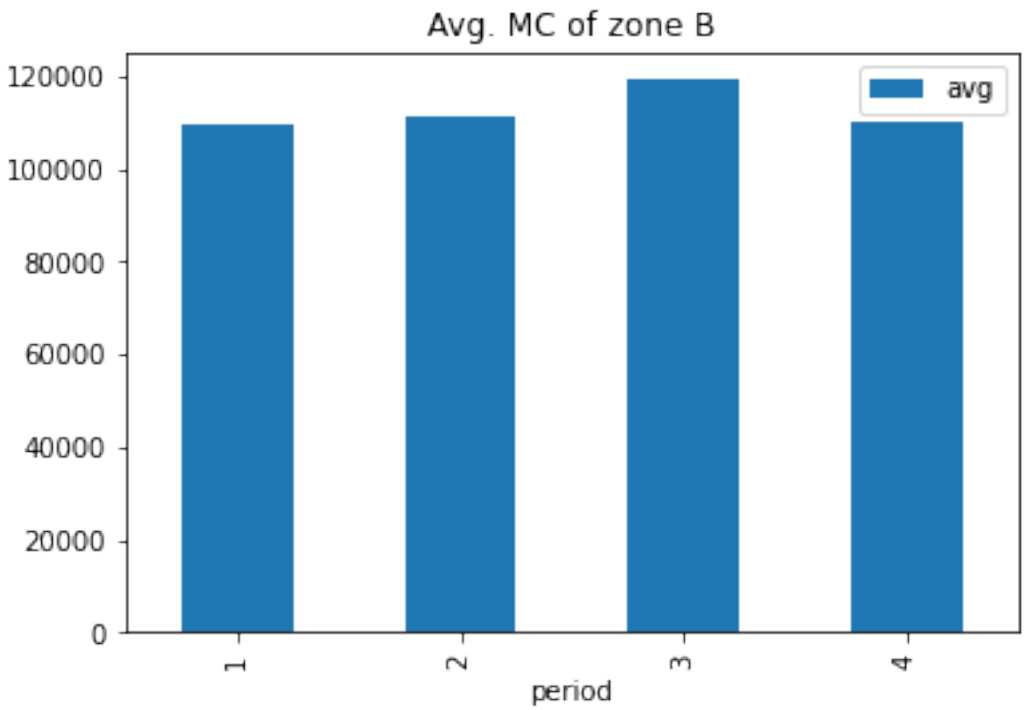
```
Relationship between each zone and their average demand per season.
```

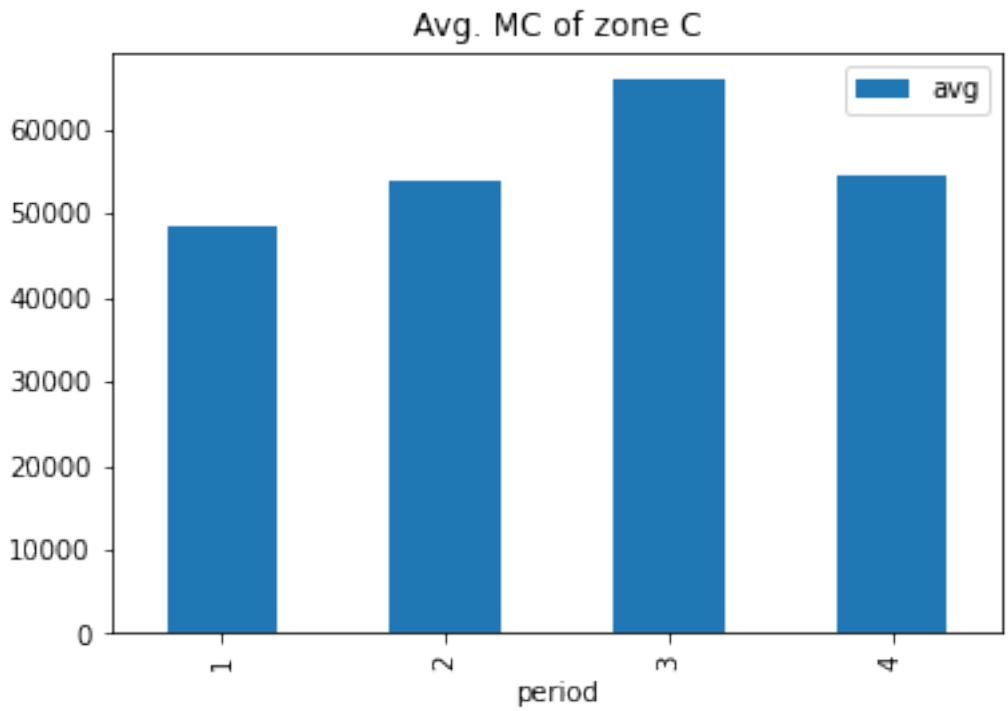


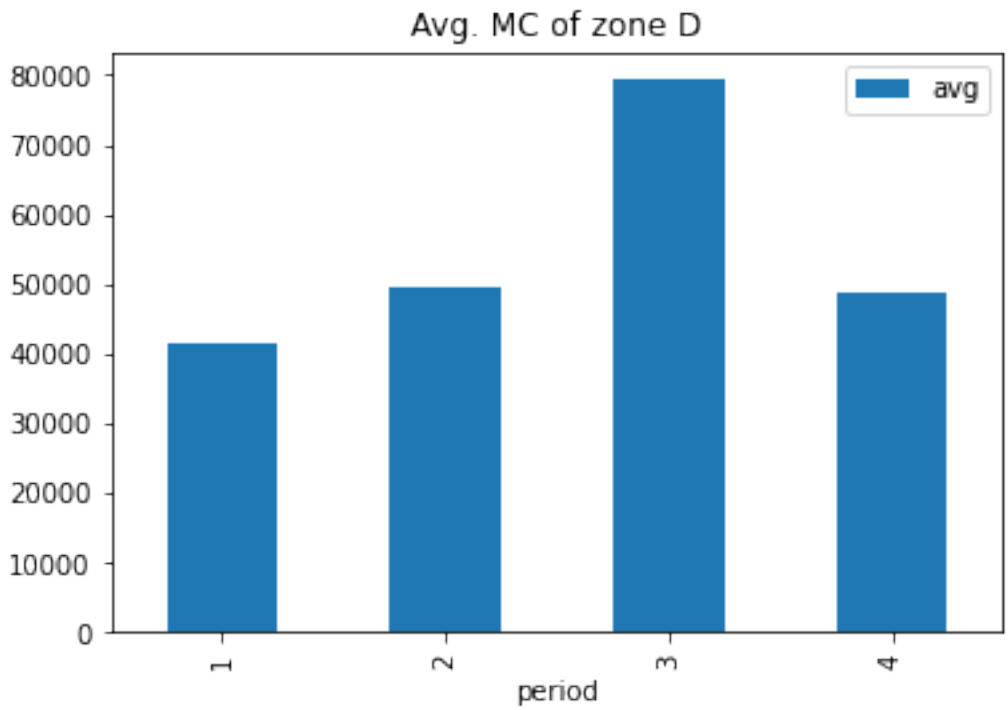
The most important information is that the Zone E has a very low demand respect to all the other zones.

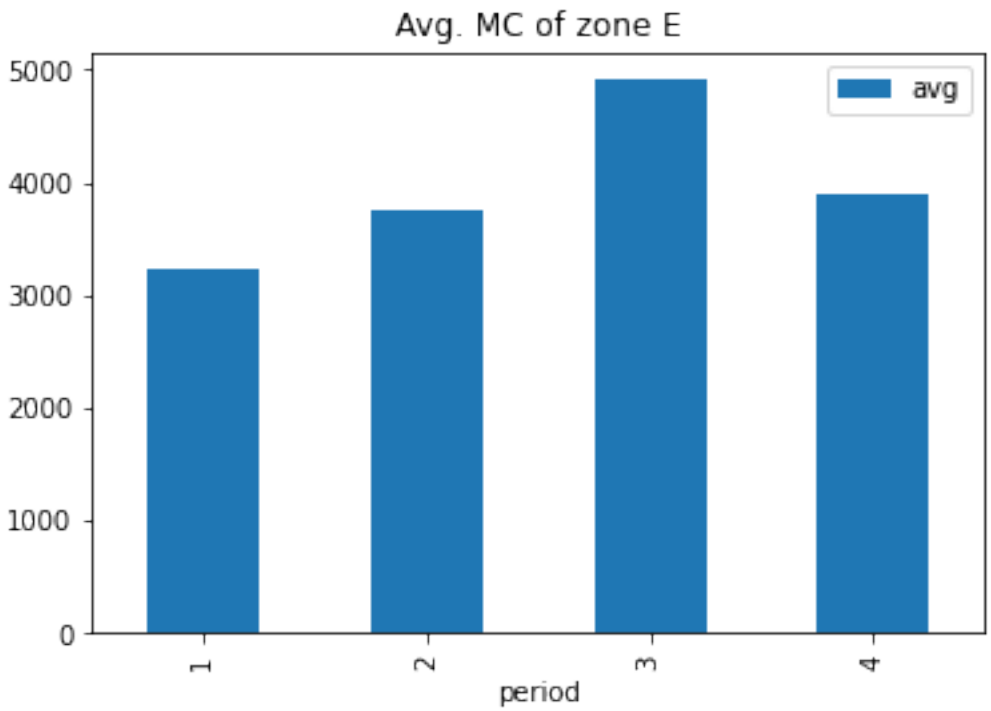
[28]:
```
for zona in database['Zona'].unique():
    MC_period[MC_period['zone'] == zona].plot(kind='bar', x='period', y='avg',␣
↪title='Avg. MC of zone '+zona)
```

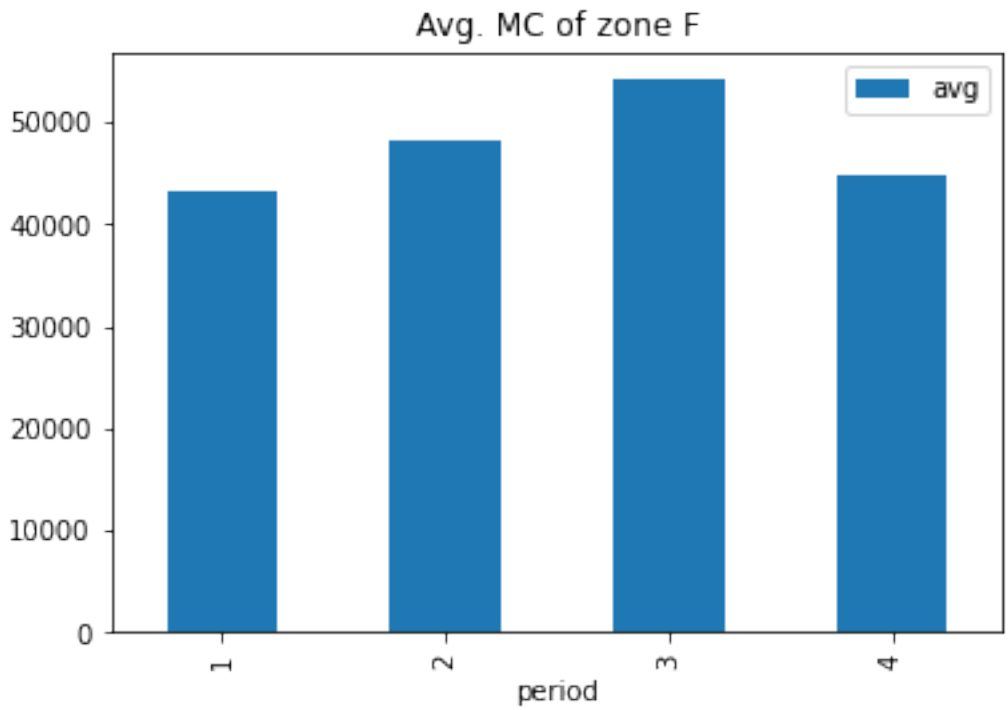

Avg. MC of zone A
avg
500000
400000
300000
200000
100000
0
1
2
3
4
period


Avg. MC of zone B
avg
120000
100000
80000
60000
40000
20000
0
1
2
3
4
period

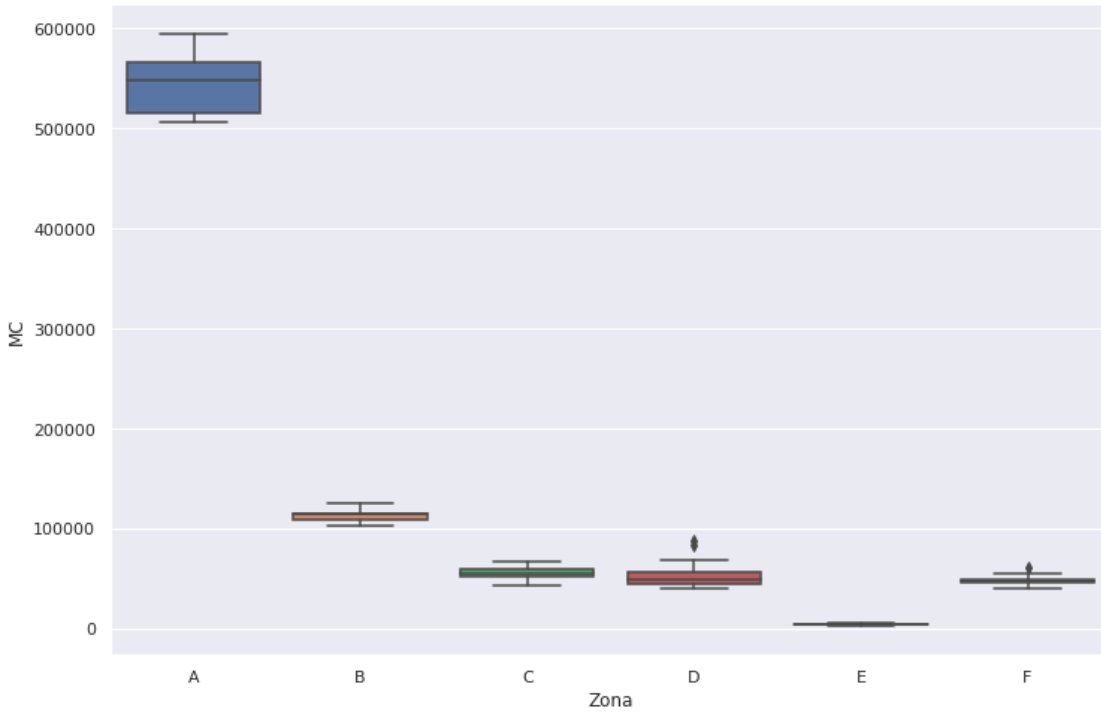

Avg. MC of zone C
avg
60000
50000
40000
30000
20000
10000
0
1
2
3
4
period


Avg. MC of zone D
avg
80000
70000
60000
50000
40000
30000
20000
10000
0
1
2
3
4
period


Avg. MC of zone E
avg
5000
4000
3000
2000
1000
0
1
2
3
4
period

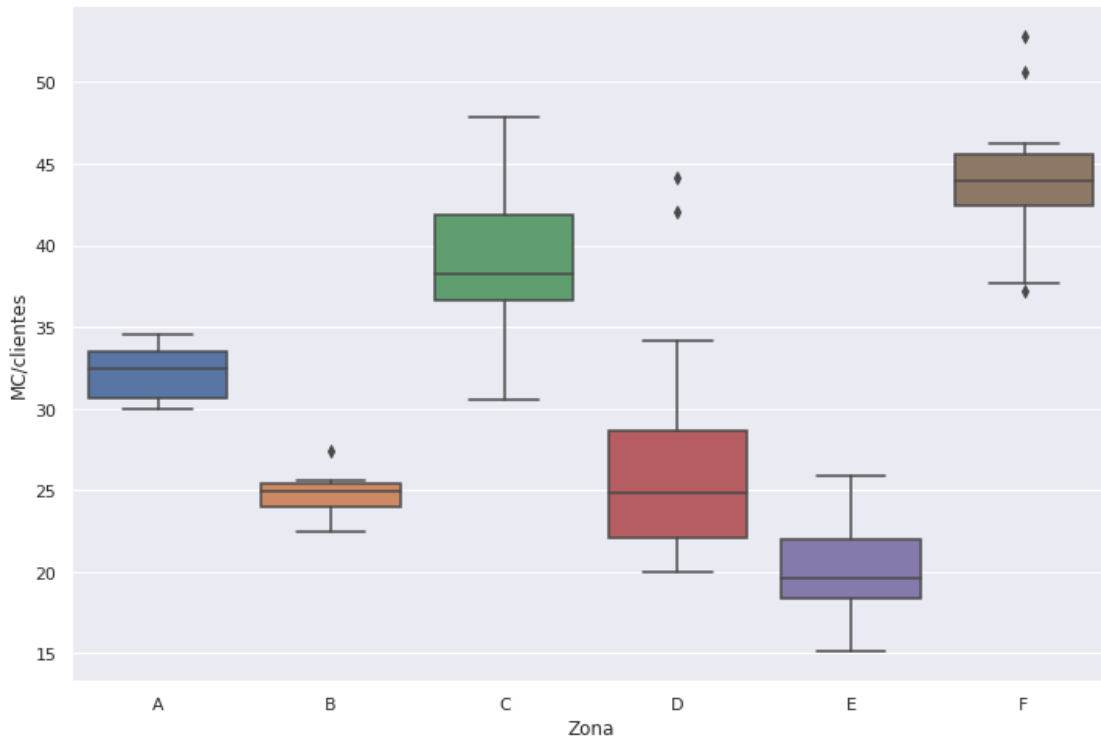

Avg. MC of zone F
avg
50000
40000
30000
20000
10000
0
1
2
3
4
period

[29]:
```
import seaborn as sns

sns.set(rc={'figure.figsize':(12,8)})
sns.boxplot(data=mc_clientes, x='Zona', y='MC');
```



[30]:
```
sns.boxplot(data=mc_clientes, x='Zona', y='MC/clientes')
```

[30]: <AxesSubplot:xlabel='Zona', ylabel='MC/clientes'>



The relationship between the distribution of the different zones changes abruptly when we analyze the demand by clients. As we noticed previously, there are big differences when we consider the total demand than considering the average demand by clients.

Variational ratio of demand

[31]:
```
variation = {}
for zona in mc_clientes['Zona'].unique():
    z = mc_clientes[mc_clientes['Zona']==zona][['MC']].values
    variation[zona] = z.std() / z.mean()

pd.DataFrame(variation, index=[0])
```

[31]:

	A	B	C	D	E	F
0	0.050699	0.057708	0.12492	0.27959	0.162567	0.116275

We can confirm a lesser variation in A and B. Also, a huge variability in zone D. Notice that in D (and F) exist outliers that could bias this information.

The previous analysis is to obtain a first insight about the data. We are checking the demand with respect to the time season and the Zone and the number of clients as principal variables.

[32]:
```
database[['MC', 'Clientes']].corr()
```

[32]:

```
                 MC  Clientes
MC        1.000000  0.994064
Clientes  0.994064  1.000000
```

[33]:

```
empresas = ['Empresas_Total', 'Empresas_Industria', 'Empresas_Construccion',␣
↪'Empresas_CTH', 'Empresas_Servicios']
database[['MC'] + empresas].corr()
```

[33]:

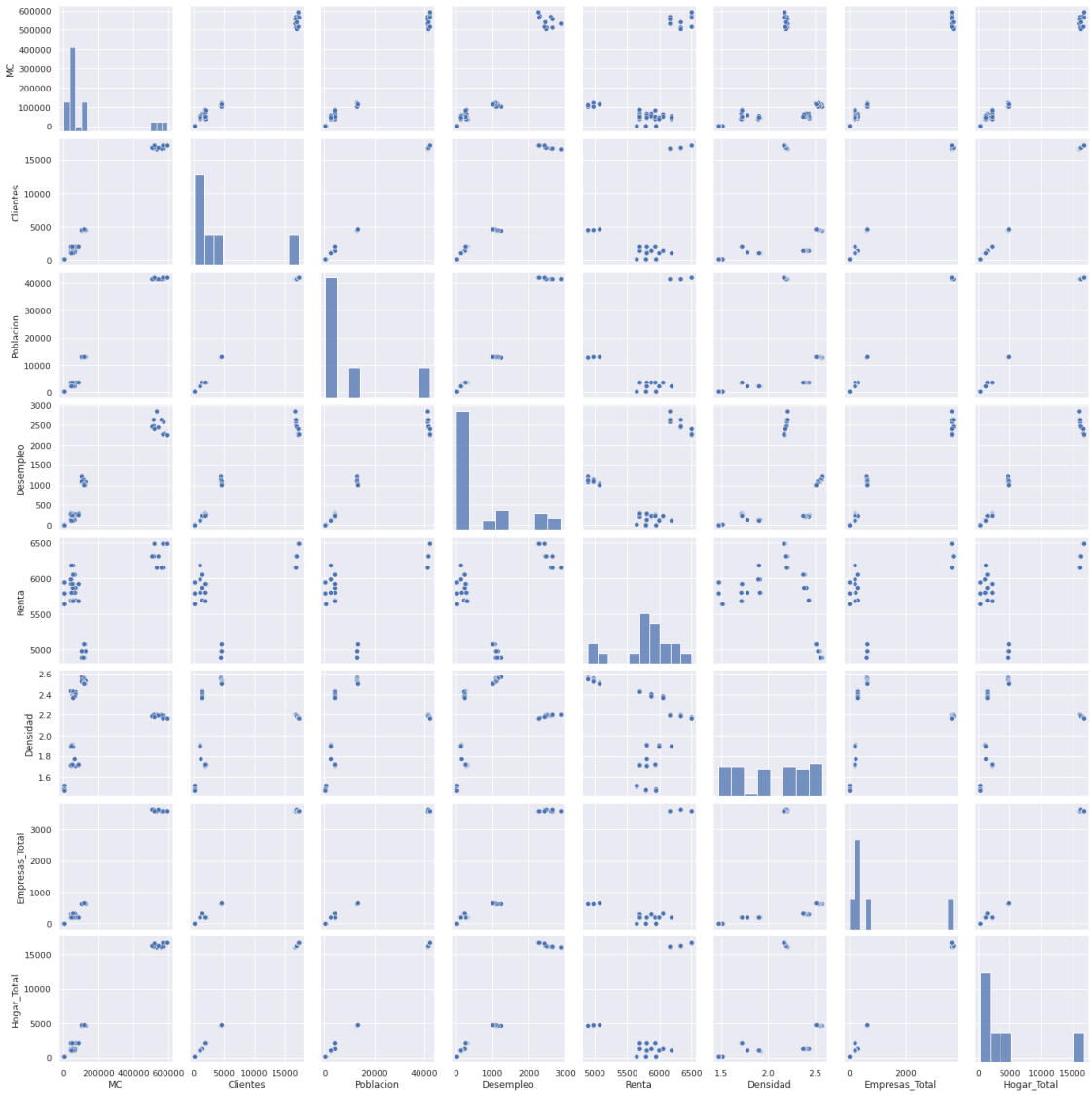
```
                            MC  Empresas_Total  Empresas_Industria  \
MC                    1.000000        0.996356            0.978583
Empresas_Total        0.996356        1.000000            0.987160
Empresas_Industria    0.978583        0.987160            1.000000
Empresas_Construccion 0.976667        0.975027            0.946625
Empresas_CTH          0.995636        0.998686            0.984292
Empresas_Servicios    0.994312        0.998073            0.982985

                      Empresas_Construccion  Empresas_CTH  Empresas_Servicios
MC                                 0.976667      0.995636            0.994312
Empresas_Total                     0.975027      0.998686            0.998073
Empresas_Industria                 0.946625      0.984292            0.982985
Empresas_Construccion              1.000000      0.980503            0.963621
Empresas_CTH                       0.980503      1.000000            0.994283
Empresas_Servicios                 0.963621      0.994283            1.000000
```

[ ]:

```

```

[34]:

```
sns.pairplot(database[['MC', 'Clientes', 'Poblacion', 'Desempleo', 'Renta',␣
↪'Densidad', 'Empresas_Total', 'Hogar_Total']]);
```



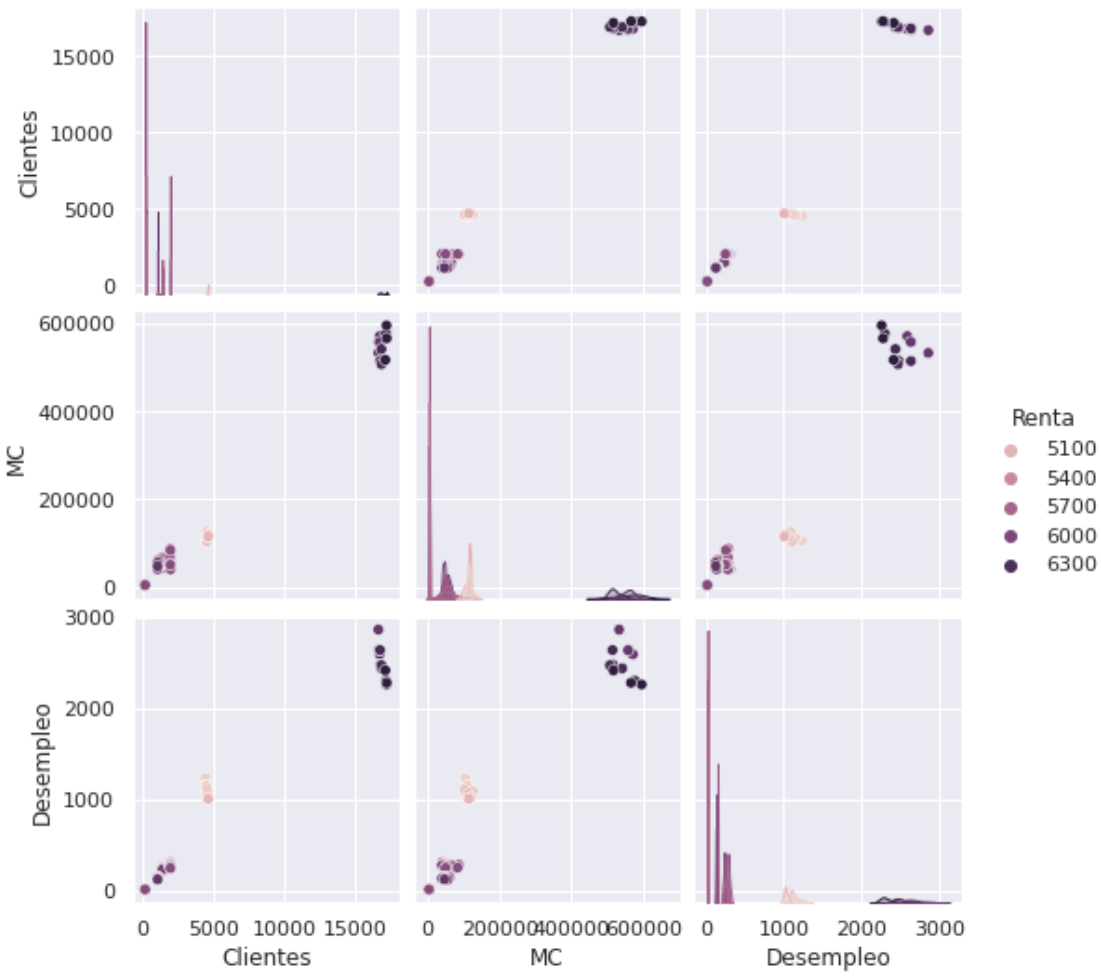
It is possible to find a positive correlation between Empresas, Hogares, Poblacion and Clientes. Which is something expectable.

Another observation is that there exists an interesting relationship with respect to rent (Renta) and the number of clients/companies/homes/population/unemployment. Rent has a similar scatter distribution with respect to all these variables. Which is also expectable given the strong correlation.

This could suggest that it is not needed to use all the variables related to population/homes/etc.

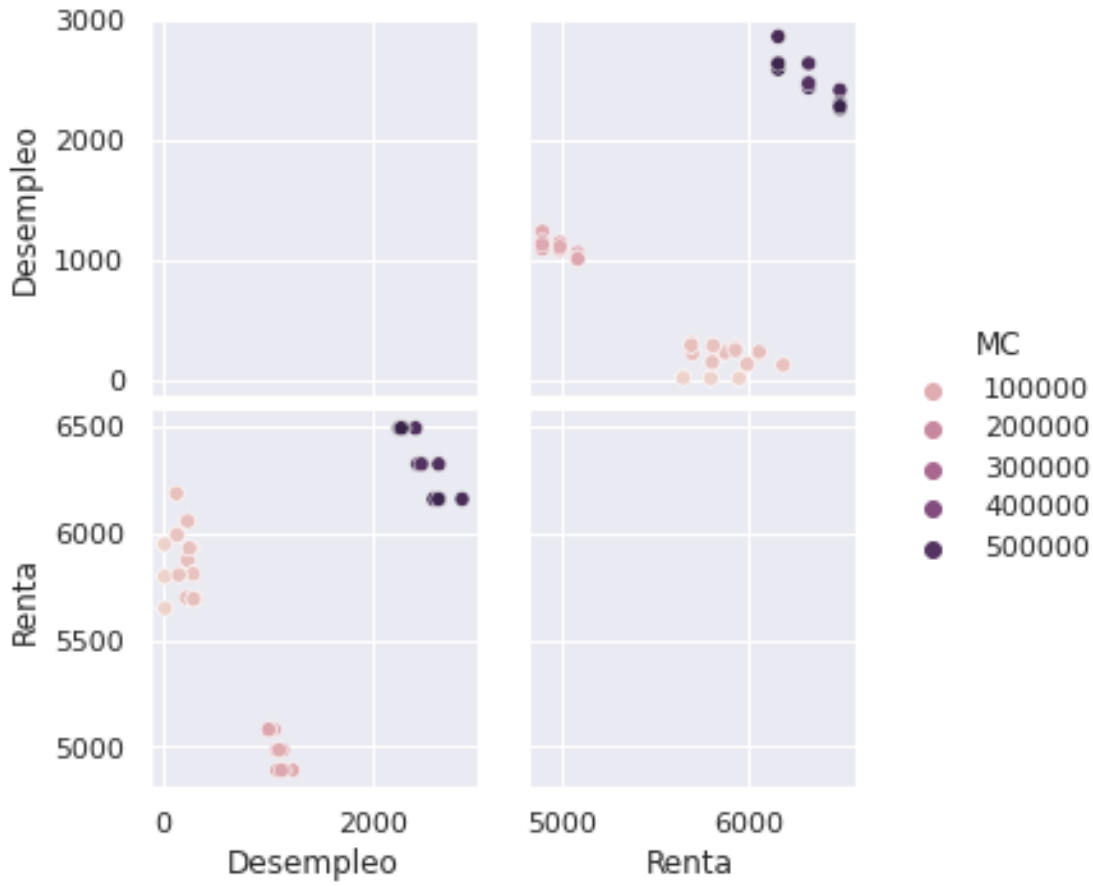
Lets see the clusters respect to the rent:

[35]:
```
sns.pairplot(database[['Clientes', 'MC', 'Desempleo', 'Renta']],
             hue='Renta');
```



There are similar relationships in terms of high rent and high consumption. Generally speaking it is possible to clusterize by rent. The only one relationship that I would like to discuss is Desempleo. I would like to understand better why high unemployment also has higher values in other variables.

[36]:
```
sns.pairplot(database[['Desempleo', 'Renta', 'MC']],
             hue='MC');
```

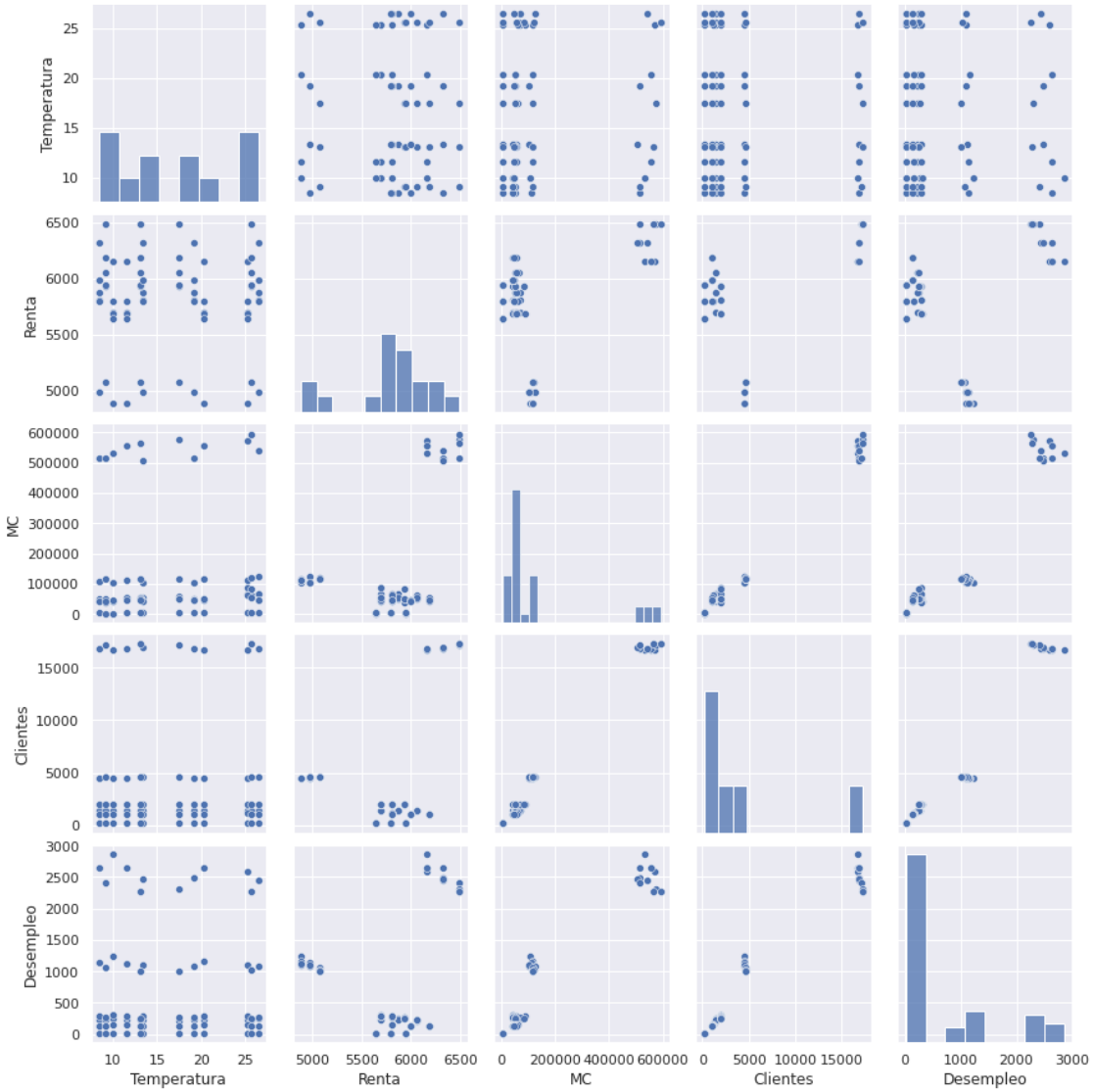


It shows that the population with high rent and with no job have higher consumption than others. Notice that it is possible to linearly separate the higher consumers.

This plot suggests that there exists a small part of the unemployed population with a high rent and high energy consumption. Also, the unemployed population with middle rent shows less consumption than the rest.

Let's check more variables with respect some of the previous (considering the correlation):

[37]:
```
sns.pairplot(database[['Temperatura', 'Renta', 'MC', 'Clientes', 'Desempleo']]);
```



I think these sets of variables could be more discriminative.

However, the number of entries is not very big. SSo we can try to use all the features. After that, with more time, we can see which is the impact of eliminating the correlated variables.

Finding a costless effort in annotation.

Moreover, it could be also compared with the use of a PCA. To see which variables are less or more informative.

MODEL

[38]:
```
X = database[['Zona', 'Fecha', 'MC']].sort_values(by='Fecha')
X
```

[38]:

```
    Zona   Fecha         MC
0      A  2019Q1  531930.18
36     D  2019Q1   41342.40
12     B  2019Q1  104350.41
24     C  2019Q1   42221.10
60     F  2019Q1   47309.73
..   …     …         …
47     D  2021Q4   49924.95
23     B  2021Q4  114164.01
11     A  2021Q4  565079.79
59     E  2021Q4    3993.30
71     F  2021Q4   46832.97

[72 rows x 3 columns]
```

As the date variable (Fecha) is in fiscal period format. We need to cast it to a numeric type. It is possible to make a total integer order from 1 to the length of the data frame. But it is not the best option in case we want to upgrade the dataframe with new intermediate entries.

A better approach could be to use a datetime ordinal converter.

[39]:

```
import datetime as dt
X = pd.concat([pd.DataFrame({'Fecha_ord': X['Fecha'] \
                                          .apply(lambda t: t.to_timestamp()) \
                                          .map(dt.datetime.toordinal)}),
               X], axis=1)

X.drop(['Fecha'], axis=1, inplace=True)
X
```

[39]:

```
     Fecha_ord Zona         MC
0       737060    A  531930.18
36      737060    D   41342.40
12      737060    B  104350.41
24      737060    C   42221.10
60      737060    F   47309.73
..         …  …         …
47      738064    D   49924.95
23      738064    B  114164.01
11      738064    A  565079.79
59      738064    E    3993.30
71      738064    F   46832.97

[72 rows x 3 columns]
```

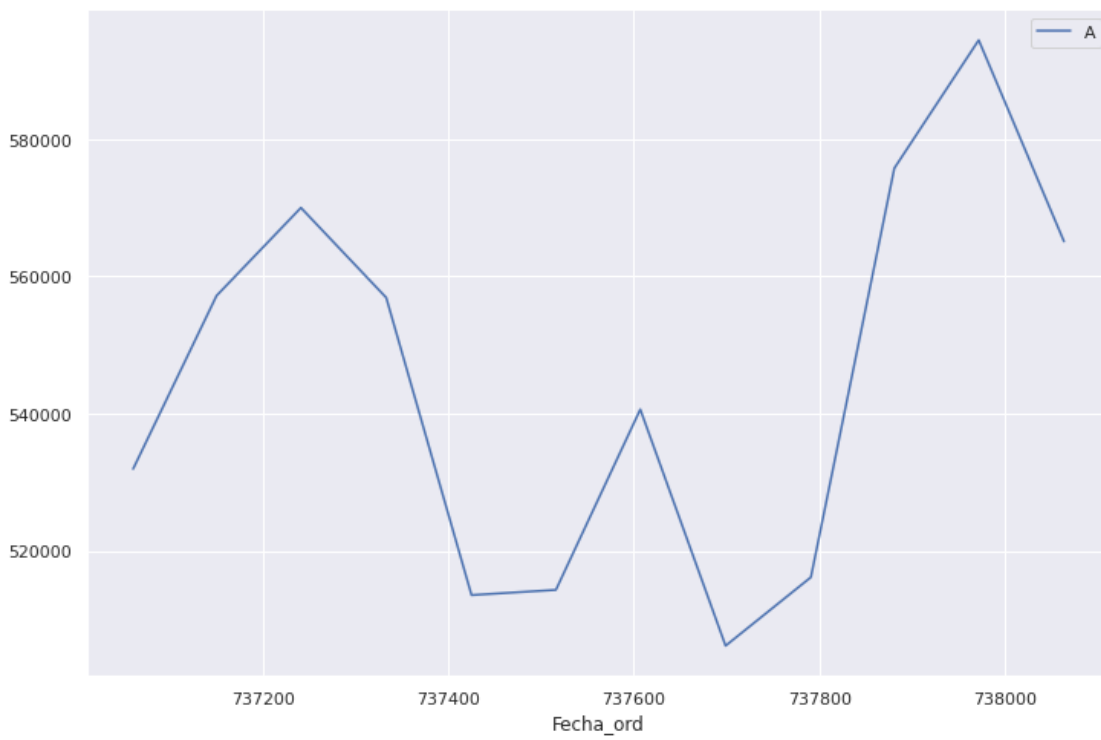
Another observation is that the prediction will be also dependent to the Zone, alternatively we can create one model for each zone. Or to create a double input (time and zone).

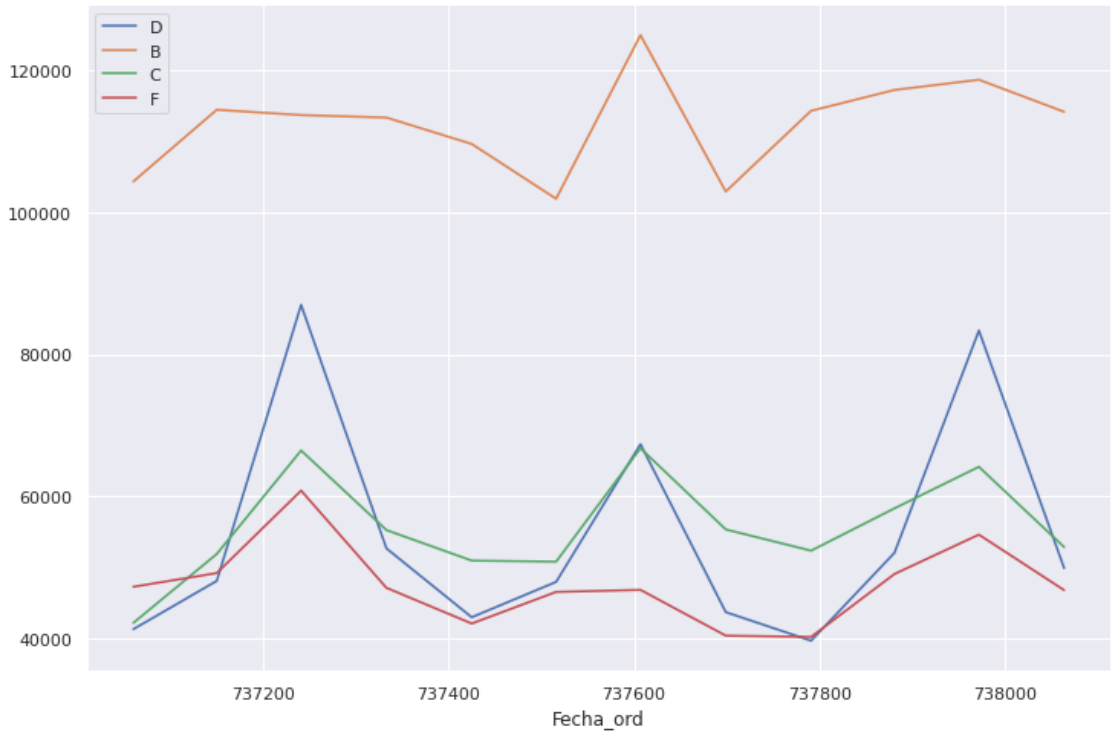
I'm considering just these variables as input because I do not have any other information to make

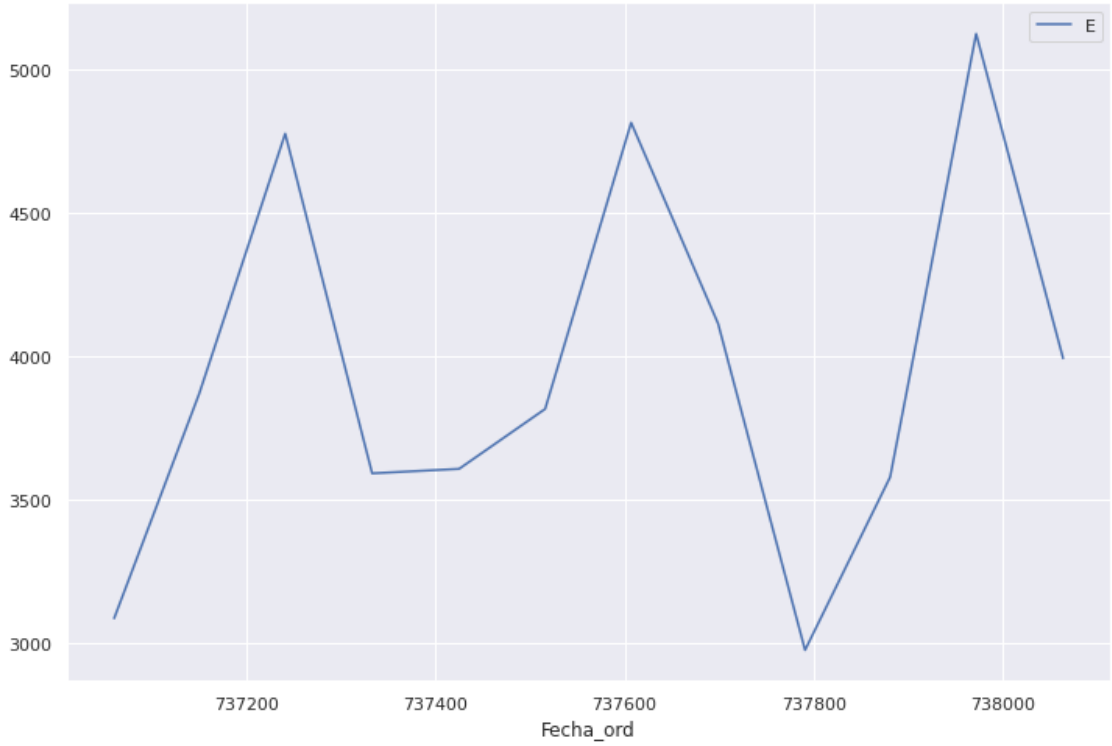
the future query. I can create the future dates and also consider all the zones to query but I can not assume any other variable like temperature, density, etc.

Another option could be to take the average of these values as an approximation. But I will need to use the same value at query time. It is not informative.

[40]:
```
# Each zone as a timeserie plot

z = X['Zona'].unique()[0]
X[X['Zona']==z].plot(x='Fecha_ord', y='MC', label=z);

z = X['Zona'].unique()[1]
ax = X[X['Zona']==z].plot(x='Fecha_ord', y='MC', label=z);
for z in X['Zona'].unique()[2:-1]:
    X[X['Zona']==z].plot(x='Fecha_ord', y='MC', ax=ax, label=z);


z = X['Zona'].unique()[-1]
X[X['Zona']==z].plot(x='Fecha_ord', y='MC', label=z);
```

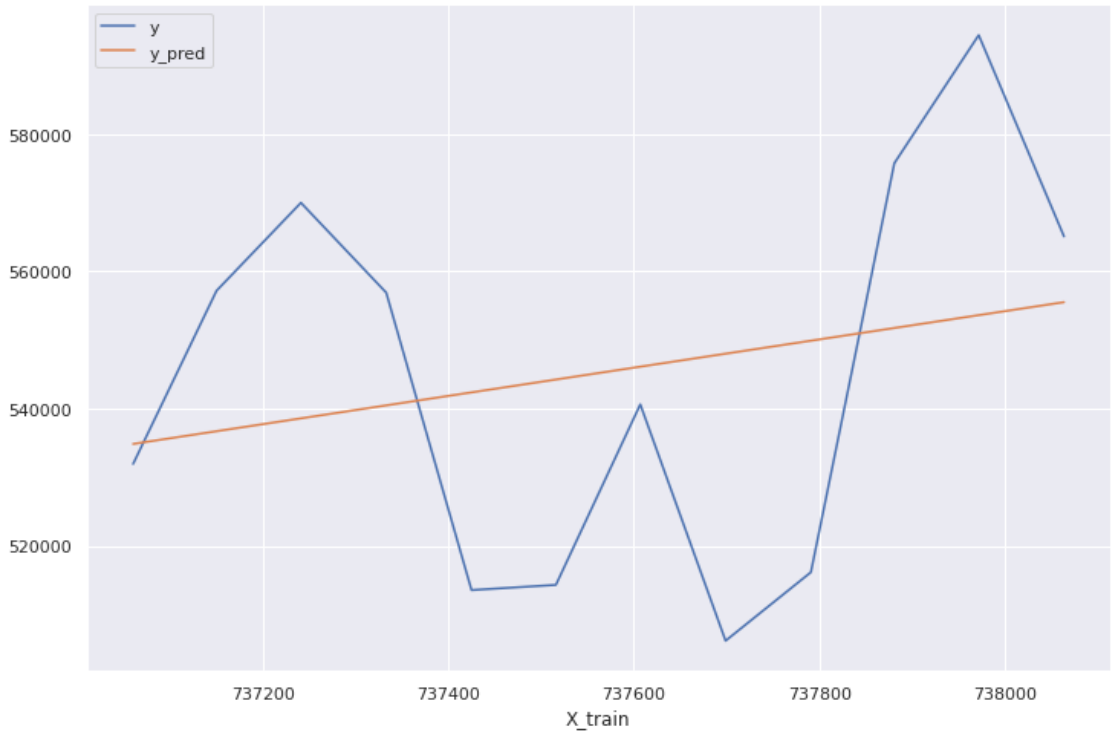


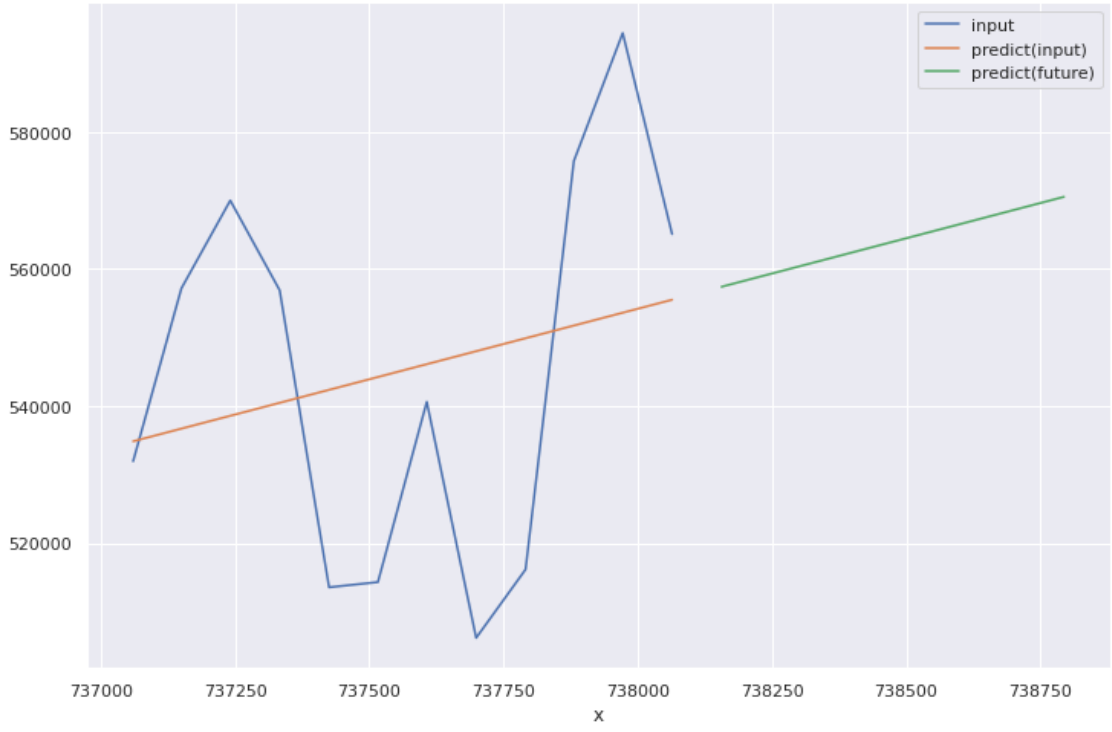

D
B
C
F
120000
100000
80000
60000
40000
737200
737400
737600
737800
738000
Fecha_ord


E
5000
4500
4000
3500
3000
737200
737400
737600
737800
738000
Fecha_ord

[41]:
```
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.svm import SVR

from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
```

[42]:
```
# Linear Regression

z='A'
X_train = X[X['Zona']==z].Fecha_ord.to_numpy().reshape(-1,1)
y = X[X['Zona']==z].MC.to_numpy()


regr = make_pipeline(StandardScaler(),
                     LinearRegression())
regr.fit(X_train, y)
y_pred = regr.predict(X_train)

result = pd.DataFrame({'X_train': X_train.squeeze(), 'y':y, 'y_pred':y_pred})
ax = result.plot(x='X_train', y='y')
result.plot(x='X_train', y='y_pred', ax=ax)


############ Future prediction ############
future = [year+period for year in ['2022', '2023'] for period in ['Q1', 'Q2',␣
↪'Q3', 'Q4']]
future_df = pd.DataFrame({'Fecha' : list(map(pd.Period, future))})

future_ord = future_df['Fecha'].apply(lambda t: t.to_timestamp()).map(dt.
↪datetime.toordinal)
future_ord


y_future = regr.predict(future_ord.to_numpy().reshape(-1,1))

result = pd.DataFrame({'X_train': X_train.squeeze(), 'y':y, 'y_pred':y_pred})
ax = result.plot(x='X_train', y='y', label='input')
result.plot(x='X_train', y='y_pred', ax=ax, label='predict(input)')

result = pd.DataFrame({'x': future_ord.to_numpy(),
                       'y': y_future})
result.plot(x='x', y='y', ax=ax, label='predict(future)');
```

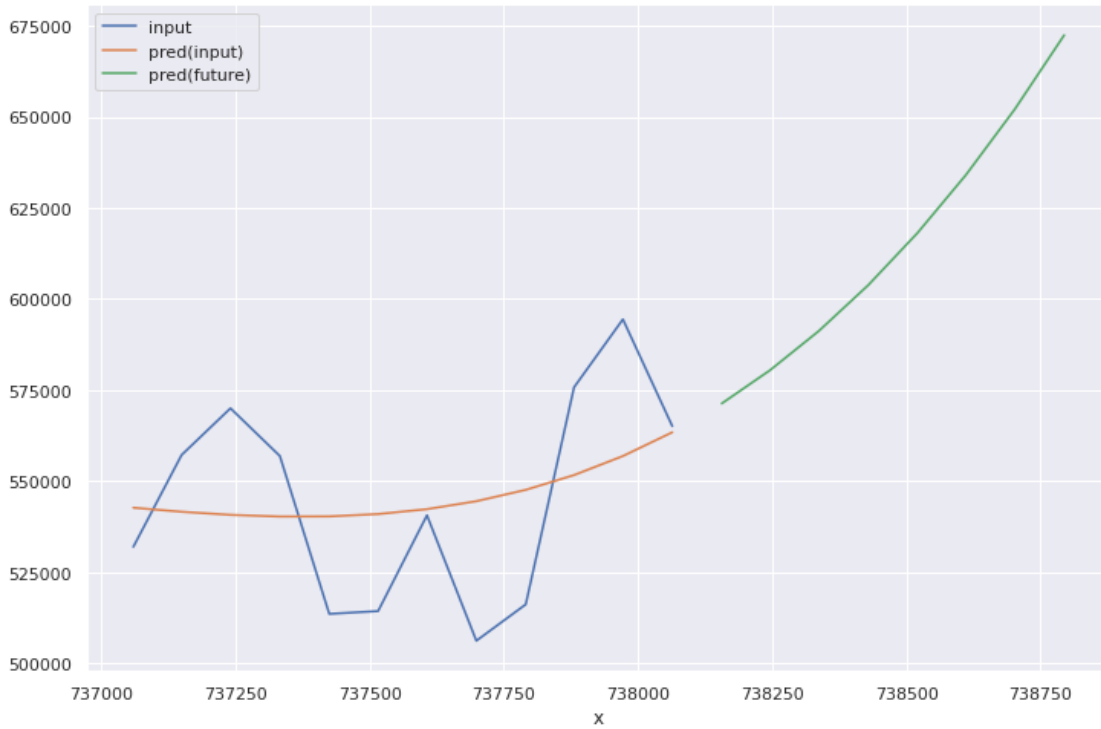

y
y_pred
580000
560000
540000
520000
737200
737400
737600
737800
738000
X_train


input
predict(input)
predict(future)
580000
560000
540000
520000
737000
737250
737500
737750
738000
738250
738500
738750
x

In this example we can see the effect of a simple linear regression. The problem with this model is its linearity. As we can see, the prediction will always increase and it is not considering the seasonal cycles.

It could be interesting to try with a nonlinear model but the number of entries for train are low. For example:

[43]:
```
# It is possible to crossvalidate with different hyperparammeters to obtaing a␣
↪more general function.

z='A'
X_train = X[X['Zona']==z].Fecha_ord.to_numpy().reshape(-1,1)
y = X[X['Zona']==z].MC.to_numpy()


regr = make_pipeline(StandardScaler(),
                     SVR(kernel="poly", C=100, gamma="auto", degree=3, epsilon=0.
↪8, coef0=3))
regr.fit(X_train, y)
y_pred = regr.predict(X_train)

result = pd.DataFrame({'X_train': X_train.squeeze(), 'y':y, 'y_pred':y_pred})
ax = result.plot(x='X_train', y='y', label='input')
result.plot(x='X_train', y='y_pred', label='pred(input)', ax=ax)


y_future = regr.predict(future_ord.to_numpy().reshape(-1,1))
result = pd.DataFrame({'x': future_ord.to_numpy(),
                       'y': y_future})
result.plot(x='x', y='y', ax=ax, label='pred(future)');
```



[44]:
```
result = {}
for z in X['Zona'].unique():
    X_train = X[X['Zona']==z].Fecha_ord.to_numpy().reshape(-1,1)
    y = X[X['Zona']==z].MC.to_numpy()


    regr = make_pipeline(StandardScaler(),
                         SVR(kernel="poly", C=100, gamma="auto", degree=2, epsilon=0.
 ↪8, coef0=3))
    regr.fit(X_train, y)
    y_pred = regr.predict(X_train)
    result[z] = regr.predict(future_ord.to_numpy().reshape(-1,1))

predictions = pd.concat([future_ord, pd.DataFrame(result)], axis=1)
predictions
```

[44]:

	Fecha	A	D	B	C	\
0	738156	552648.707794	50164.741225	115064.455508	56078.450763	
1	738246	553527.288372	50408.971732	115252.679194	56123.649441	
2	738337	554499.161976	50666.408567	115443.210073	56119.317764	
3	738429	555567.097591	50937.399553	115636.055298	56063.797192	
4	738521	556720.881856	51219.174321	115829.122329	55956.855867	
5	738611	557932.653377	51505.258236	116018.206877	55802.483213	

```
6  738702  559241.420047  51805.013517  116209.608182  55596.362744
7  738794  560649.950849  52118.787987  116403.333399  55336.835922

               F            E
0  46890.258486  3991.926026
1  46966.744267  3980.871625
2  47065.423193  3960.716962
3  47187.002778  3931.164441
4  47330.517853  3892.385399
5  47492.138420  3845.521542
6  47676.898076  3789.159540
7  47885.504332  3723.001798
```

Conclusion

I created this predictive model to provide a complete pipeline but it is not suitable for different reasons.

The model is very simple considering we do not have an enough amount of data to create a more complex one.

Another option that could be interesting to explore is the use of all the information provided as input (including the zones with a One-hot encoding). The disadvantage of this approach is that we will need to assume the context of the future. So it is not applicable.

As the amount of data is not big and the linear model suffers monotony, in my opinion, it could be better to use an indicator (avg + std) instead of a time series forecasting approach. Unless we are able to find an extensive history of energy consumption.

A middle-step option is to use a simple model combined with a windows indicator. But, one more time, we need more data.

0.2 Question 2

[45]:
```
import openrouteservice as ors
import folium
```

[46]:
```
# Names and coordenates for each place of interest
places={'camp_nou': [41.381379, 2.123506],
        'santiago_bernabeu' : [40.453250, -3.688344]}

# Middle point of all the coords. To set the map at the center.
middle_place = np.average(np.array(list(places.values())), axis=0)

map_canvas = folium.Map(location=middle_place, tiles='cartodbpositron',␣
 ↪zoom_start=7)

coordinate = list(map(lambda x: list(reversed(x)), places.values()))
```

```
ors_key = '' #ADD YOUR OWN KEY
client = ors.Client(key=ors_key)

iso = client.isochrones(
    locations=coordinate,
    range_type='time',
    range=[900, 1800],
    attributes=['area']
)

for isochrone in iso['features']:
    folium.Polygon(locations=[list(reversed(coord)) for coord in␣
 ↪isochrone['geometry']['coordinates'][0]],
                   fill_color='blue',
                   stroke=True,
                   color='blue',
                   opacity=0.5).add_to(map_canvas)

for name, coord in places.items():
    folium.Marker(coord, tooltip=name).add_to(map_canvas)

map_canvas
```

[46]: <folium.folium.Map at 0x7fee739e3a60>

0.3 Question 3

Write a function in Python which takes a vector of length L consisting only of 0s and 1s as input and returns the largest number of consecutive 1s from the vector as an integer.

[47]:
```
def count_approach(input_):
    count, i, res = 0, 0, 0

    input_.append(0)  #add extra zero to ensure comparison
                      #in case of max number of 1s at the end. OK, it is not␣
 ↪elegant, but actually works.
    while i < len(input_):
        if input_[i] == 1:
            count += 1
        else:
            res = max(res, count)
            count = 0
        i += 1

    return res
```

```python
# similar but shorter.
def count_approach2(input_):
    count, i, res = 0, 0, 0

    input_.append(0)

    for x in input_:
        count += (x == 1)
        if x == 0:
            res = max(res, count)
            count = 0
    return res


if __name__ == "__main__":
    in1 = [1,1,0,1,1,1,0,0,1]
    in2 = [0,0,0,1,0,0,1,1]
    in3 = [1,1,1,1]
    in4 = [1]

    for input_ in [in1, in2, in3, in4]:
        print(count_approach(input_))
        print(count_approach2(input_))
```

```
3
3
2
2
4
4
1
1
```

0.4 Question 4

Understanding the problem (first insights) In this section, I'll be generating synthetic data to obtain a first insight about the problem.

[48]:
```python
def generate_uniform_data(ul=100, N=100):
    return [np.random.uniform(0,ul+1) for x in range(N)]

def syn_data_generator(ul, N):
    sql = generate_uniform_data(ul, N)
    code = generate_uniform_data(ul, N)

    data = pd.DataFrame({'sql': sql,
                         'code':code,
                        })
```

```
    return data
```

[49]:
```
# example of a dataframe with 100 entries (for both sql and code scores).
data = syn_data_generator(ul=100, N=100)

# Filtering the rows with a sum greater than 100.
data_invited = data[data.sum(axis = 1) > 100]
len(data_invited)
```

[49]: 55

Which is the average number of invited candidates (sum > 100) considering different amount of candidates (N). Meaning, the convergence of the conditional variable $[U_{code}+U_{sql}]$ when the number of candidates increase.

[50]:
```
n_experiments = 10
ul=100
data100 = [syn_data_generator(ul=ul, N=100) for x in range(n_experiments)]
data1000 = [syn_data_generator(ul=ul, N=1000) for x in range(n_experiments)]
data10000 = [syn_data_generator(ul=ul, N=10000) for x in range(n_experiments)]
```

[51]:
```
# Intentionaly blank.
```

[52]:
```
data_invited100 = [len(data[data.sum(axis = 1) > 100]) for data in data100]
data_invited1000 = [len(data[data.sum(axis = 1) > 100]) for data in data1000]
data_invited10000 = [len(data[data.sum(axis = 1) > 100]) for data in data10000]
```

[53]:
```
experiments = pd.DataFrame({'100': data_invited100,
                            '1000': data_invited1000,
                            '10000': data_invited10000,
                           })
experiments.describe()
```

[53]:

	100	1000	10000
count	10.000000	10.000000	10.00000
mean	51.700000	514.800000	5095.50000
std	6.976946	10.538817	30.67844
min	40.000000	502.000000	5046.00000
25%	46.500000	505.500000	5075.75000
50%	53.500000	513.500000	5092.50000
75%	55.750000	521.750000	5116.00000
max	62.000000	534.000000	5143.00000

The average amount of invited candidates (mean of the variable $[U_{code} + U_{sql}]_{>100}$) converges to $N/2$.

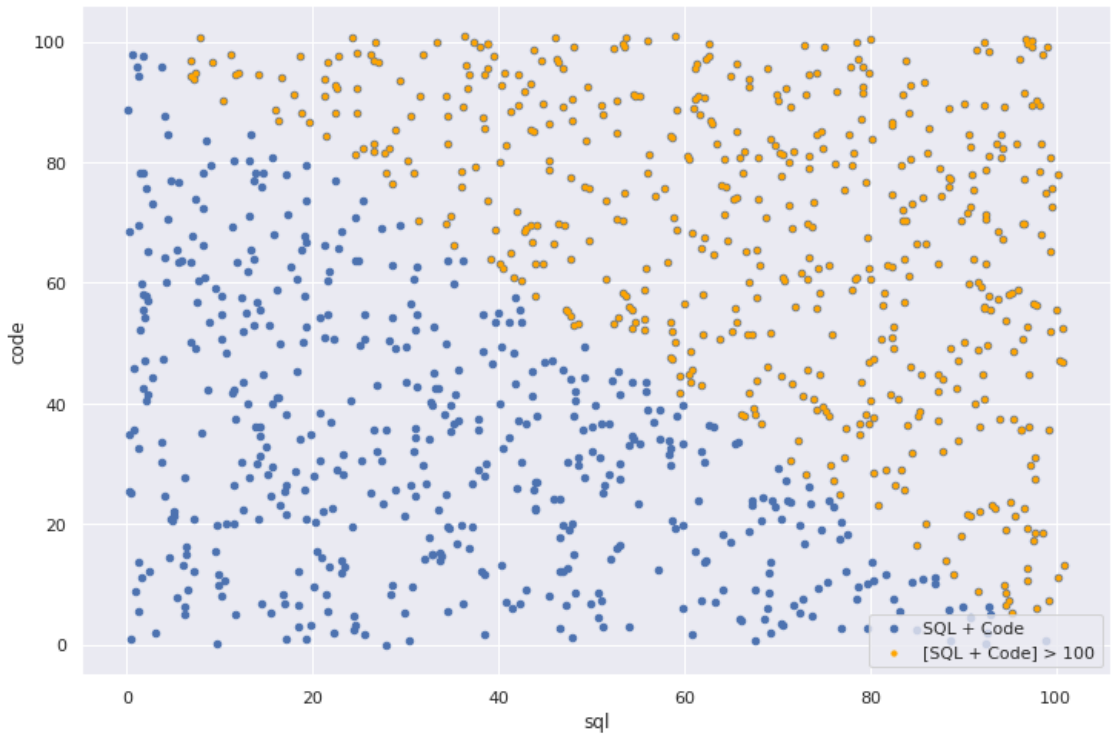
Which means that considering this 'sum score' **we are filtering the half part of the candidates**.

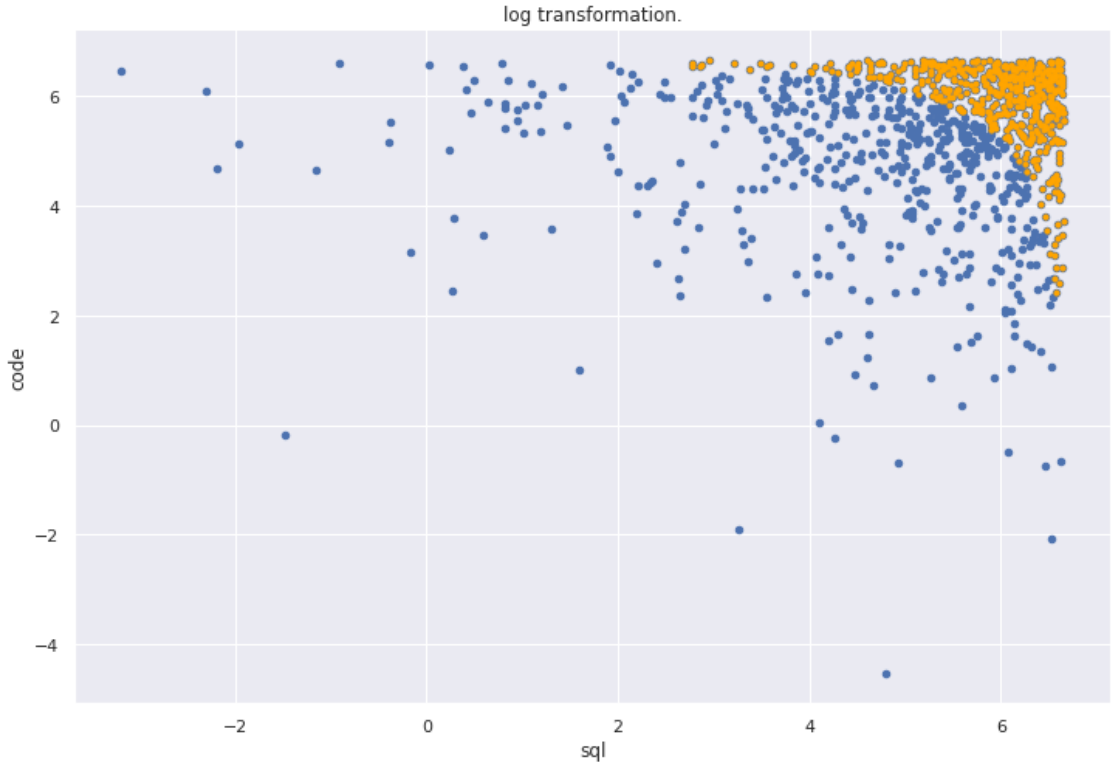
Plots and correlation analysis.

[54]:
```
def plot_data(data, data_invited):
    ax = data.plot(kind='scatter', x='sql', y='code', s=20, label='SQL + Code');
    data_invited.plot(kind='scatter', x='sql', y='code', c='orange', s=10,␣
 ↪label='[SQL + Code] > 100', ax=ax);

    ax = np.log2(data).plot(kind='scatter', x='sql', y='code', s=20, title='log␣
 ↪transformation.');
    np.log2(data_invited).plot(kind='scatter', x='sql', y='code', c='orange',␣
 ↪s=10, ax=ax);
```

[55]:
```
ul = 100
N = 1000

data = syn_data_generator(ul, N)
data_invited = data[data.sum(axis = 1) > ul]

plot_data(data, data_invited)

print(data.corr())
print(data_invited.corr())
print('\n------log transformation-------\n')
print(np.log2(data).corr())
print(np.log2(data_invited).corr())
```

```
           sql      code
sql   1.000000  0.025006
code  0.025006  1.000000
           sql      code
sql   1.000000 -0.484864
code -0.484864  1.000000

------log transformation-------

           sql      code
sql   1.000000  0.025879
code  0.025879  1.000000
           sql      code
sql   1.000000 -0.411208
code -0.411208  1.000000
```


100
80
60
40
20
0
code
0
20
40
60
80
100
sql
SQL + Code
[SQL + Code] > 100


log transformation.
6
4
2
0
−2
−4
code
−2
0
2
4
6
sql

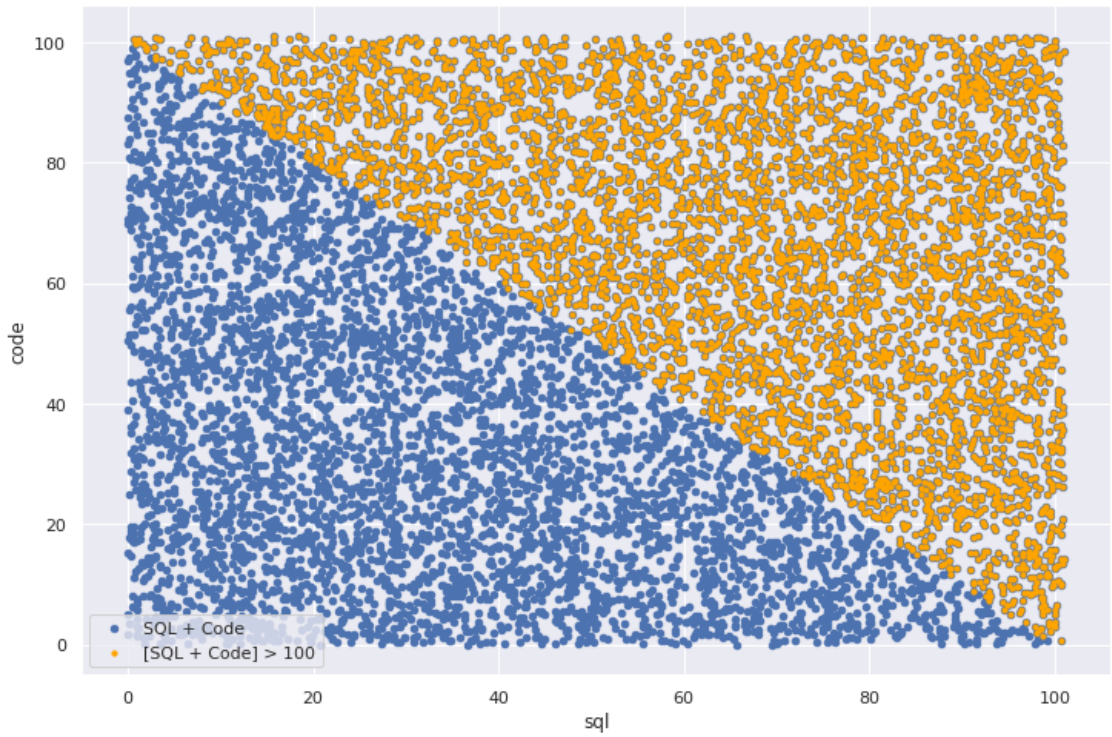
[ ]:

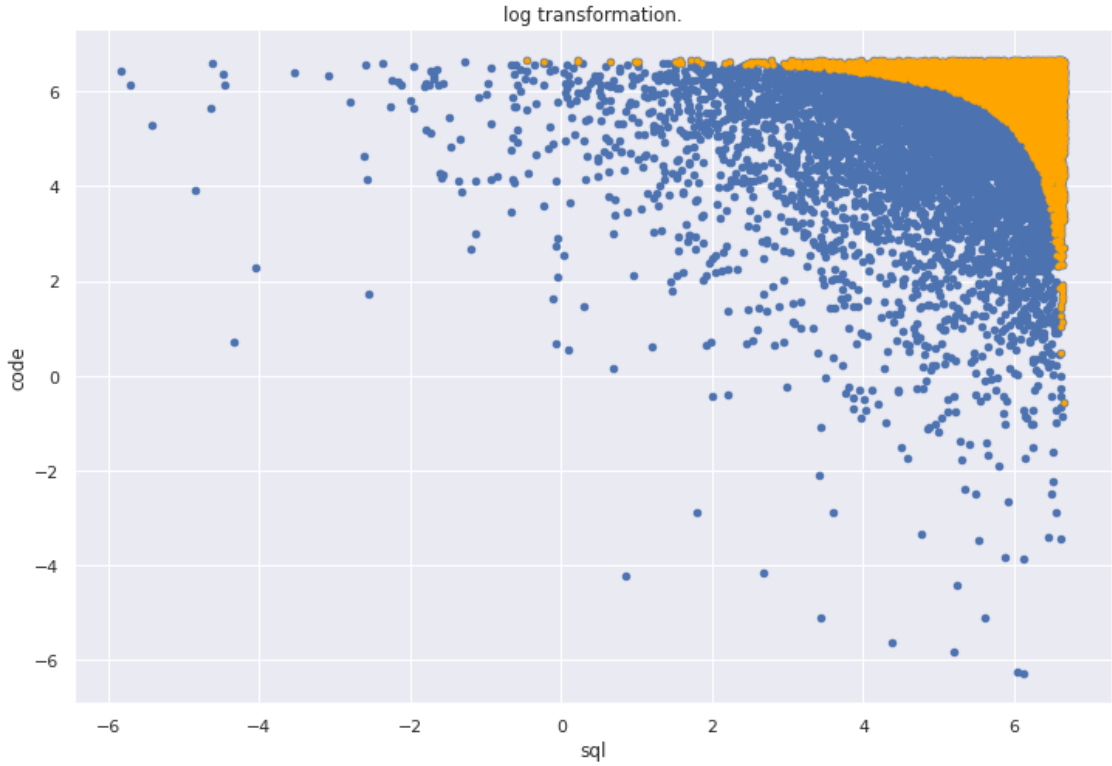
```

```

[56]:

```
ul = 100
N = 10000
data = syn_data_generator(ul, N)
data_invited = data[data.sum(axis = 1) > ul]
plot_data(data, data_invited)

print(data.corr())
print(data_invited.corr())
print('\n------log transformation-------\n')
print(np.log2(data).corr())
print(np.log2(data_invited).corr())
```

```
           sql      code
sql   1.000000  0.011048
code  0.011048  1.000000
           sql      code
sql   1.000000 -0.484291
code -0.484291  1.000000

------log transformation-------

           sql      code
sql   1.000000  0.005615
code  0.005615  1.000000
           sql      code
sql   1.000000 -0.403904
code -0.403904  1.000000
```


100
80
60
40
20
0
code
0
20
40
60
80
100
sql
SQL + Code
[SQL + Code] > 100


log transformation.
6
4
2
0
−2
−4
−6
code
−6
−4
−2
0
2
4
6
sql

Seeing the plots, I'm not able to confirm there is a correlation between the variables. Checking the Pearson's coefficient, we can find a very low correlation between the U_{sql} and U_{code} variables. But it is possible to notice a higher correlation comparing the variables that sum more than 100.

However, I am not convinced to say that the correlation is strong. Actually, when the number of entries increases the correlation looks lower (it is possible to check when N goes from $10e2$ to $10e3$). Which makes me think that taking a limit when the number of candidates increases the correlation is between 0 and 0.5.

It could be needed a more extensive simulation. But as a general answer I can say:

1. There is no correlation between SQL and CODE scores.
2. Considering the conditional sumatory (sql+code > 100) it is possible to find a "middle" or "low" correlation. Meaning, that there exists more correlation than the complete SQL and CODE scores but it is not strong.

For a sanity check, we can simulate different datasets increasing their size (CTL). Then, plot the correlation with respect to the dataset size and see if there is a convergence to 0 and a convergence to +-0.5 in the $sumScore$ and $sumScore > 100$ respectively.

[58]:
```
ul = 100
N = [10, 10, 100, 1000, 10000]

data_corrs, data_invited_corrs = [], []
for n in N:
    data = syn_data_generator(ul, n)
    data_invited = data[data.sum(axis = 1) > ul]

    data_corrs.append(data.corr().code.sql)
    data_invited_corrs.append(data_invited.corr().code.sql)

ax = pd.DataFrame(data_corrs, N).plot(label='data correlation');
pd.DataFrame(data_invited_corrs, N).plot(ax=ax, label='data_invited␣
↪correlation');
```

Finally, we obtained NO CORRELATION for $U_{sql} + U_{code}$ and a middle correlation for $[U_{sql} + U_{code}]_{>100}$

A very important fact is that the previous analysis is more accurate when the number of candidates increases enough. However, It is suitable. In other cases (a few candidates) it is possible to analyze them case by case.

Meaning that the approach could be a good attempt to make a "middle-weak" filter to consider the half part of the candidates.